

现代交换原理仿真实验系统

——《交换技术与交换网》课程辅助实验系统

北京邮电大学计算机学院开设的“现代交换原理”课程是第一个在国内院校开设的交换类课程，该课程2004年被评为北京市精品课程，2005年被评为国家级精品课程。

交换技术是通信网的核心技术，交换课程是通信与信息类专业具有特色的、必不可少的专业基础课程，通过本课程的学习，可使学生深刻理解和掌握交换的基本原理和各种技术，为更深入的专业学习打下基础。为了配合“现代交换原理”课程的教学，使学生进一步消化吸收课堂上所学的理论知识，加深对各种交换方式以及呼叫处理基本原理的理解，掌握各种实用交换技术的基本原理和方法，我们依托2001年北京市教委教改项目，研制实现了“现代交换原理仿真实验系统”，经多届学生使用，取得了良好的效果。

该仿真实验系统无需各类昂贵的通信设备，在最普及的微机系统上即可开设。实验的设计思想采用“嵌入－替换的方法”，使学生灵活、直观、方便地进行覆盖各知识点的多种实验，具有交互性强、实时动态检错、技术含量高（涉及各种交换技术和通信网复杂协议）的特点。任课教师可根据教学安排，适当选取实验指定学生完成，以配合课堂教学内容的掌握。

该仿真实验系统（版本v1.1）主要包括两个实验模块：基础型实验模块和提高型实验模块。

基础型实验模块主要涉及电路交换的基本原理和技术（程控数字交换原理与技术），该模块包含六个编程实验：时间表调度实验、摘挂机检测实验、脉冲计数实验、位间隔识别实验、软件送音实验、驱动交换网络实验。

提高型实验模块主要包含分组交换实验、ATM交换实验和MPLS（多协议标记交换）实验。

分组交换实验包含有：分组交换演示实验、永久虚电路建立演示实验和分组交换编程实验，而分组交换编程实验有3个，它们是：呼叫建立实验、呼叫拆除实验、分组转发实验。

ATM交换实验包含有：ATM交换演示实验、ATM路由表查询实验。

MPLS交换实验包含有：MPLS演示实验、标记转发表实验、MPLS编程实验。

对于编程实验，实验方法采用“嵌入—替换的方法”。要求学生做的各种交换原理实验，是以模块的形式提供的。学生根据所给相应实验的模块名和与其它各模块的接口（参数和数据结构），按照实验指导编程，完成指定的功能，然后将仿真交换控制软件内的同名模块替换下来，这样在交换控制软件内就嵌入了学生所编的要求完成指定功能的模块。如果整个系统能完成前述的通过微机键盘操作进行的呼叫（通信），并且交换控制过程的显示正常，则表明学生所编模块正确，实验通过。

版权所有：北京邮电大学计算机科学与技术学院-摩托罗拉联合实验室

如遇侵权请垂询：jlbian@bupt.edu.cn

交换仿真实验平台简介(一)

链接：[实验平台概述](#)

链接：[有关实验平台的视图](#)

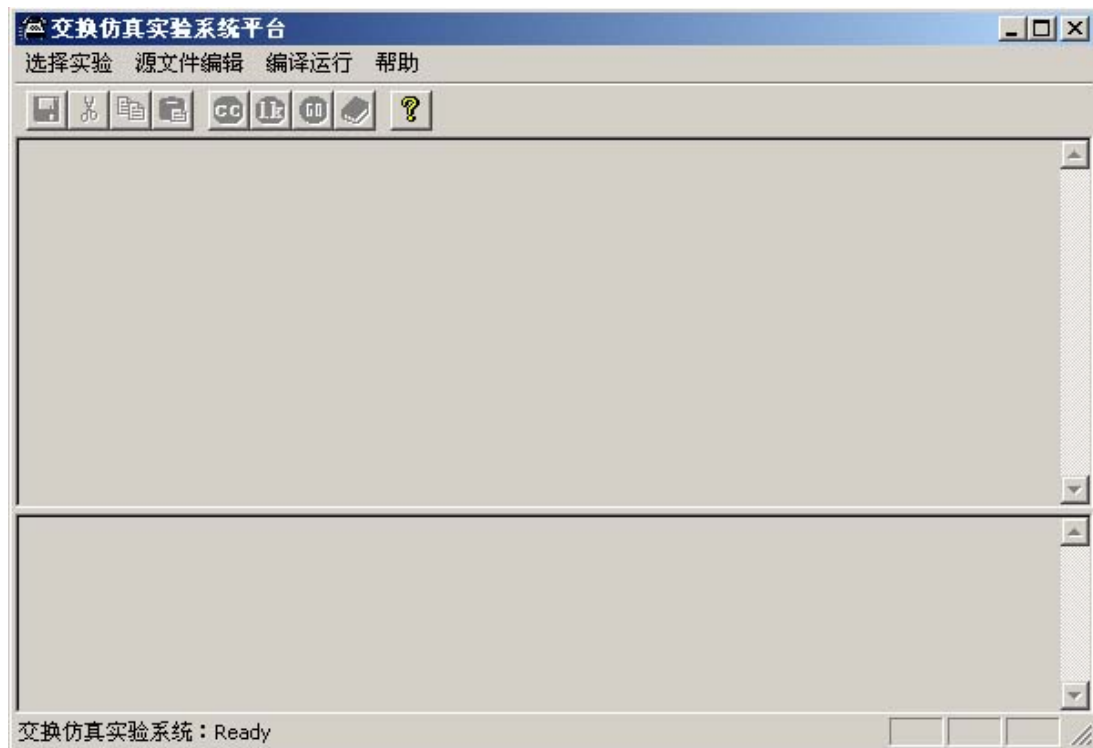
链接：[有关实验平台的菜单](#)

链接：[有关实验平台的工具栏和状态栏](#)

链接：[有关实验系统的目录设置](#)

一.实验平台概述：

下图即交换仿真实验系统平台的主界面，因为该实验系统中多数的实验需要编程实现,我们的主界面采用了像Visual Studio、Dephi这样的集成开发环境界面，集成了需编程实验需要的编辑、编译、链接、运行、演示以及帮助等常用的工具。



下面我们对上面的实验环境进行简单的介绍，以帮助大家尽早地熟悉该实验环境。

二. 有关实验平台的视图：

本实验系统包括两个视图：编辑视图和结果视图。

编辑视图：在主界面上半部分的视图为编辑视图，主要用于编辑源代码使用；

结果视图：在主界面下半部分的视图为结果视图，主要用于显示源代码编译、链接的结果信息；

两个视图都含有垂直滚动条，并且两个视图的相对大小可以手工调节。

三. 有关实验平台的菜单：

在主菜单中包括“选择实验”、“源文件编辑”、“编译运行”、“帮助”几个部分。

1.在“选择实验”菜单下包括基础实验（程控交换实验）、提高实验（分组实验、ATM实验、MPLS实验）的相关实验项，点击相应菜单项后可以进入该实验；我们也可在一个实验下通过“选择实验”菜单切换到其他的实验；退出实验的“退出”选项也在该菜单项（退出该实验环境也可以通过点击标题栏的“关闭”按钮实现）；

2.在“源文件编辑”菜单下（在需编程实验中可用）包括了常用的编辑命令：撤销、剪切、复制以及粘贴等。因为该菜单主要针对需编程实验，所以保存源代码文件的命令也集成在了该菜单项下。

这里要强调的是当您点击“保存”命令时，我们会把视图内的内容自动保存在您安装目录下的sourcefile目录下。当下一次您重新开启该实验系统时，该目录下的文件将会重写,所以，如果您需要备份源文件时，我们建议两种办法：

- 1.在实验过程中，把您需要备份的文件另存于其他目录下；
- 2.在实验结束后，把您安装程序的“sourcefile”子目录下需要备份的文件另存于他处。

交换仿真实验平台简介(二)

3.在“编译运行”菜单下包含了“编译”、“连接”、“运行”、“演示”四个菜单项，它们的用途如下：

编译菜单项：编译 (Compile)当前视图中的源代码文件，并把编译后的结果显示在结果视图下面。当源代码文件没有语法错误时，结果视图内会显示产生的对象文件 (.obj文件)；当源代码文件存在语法错误时,结果视图内会显示出错的行号和出错原因。该菜单项只在需编程实验中可用。

链接菜单项：链接 (Link)当前目录下的对象文件 (.obj文件) 成动态链接库文件 (.dll文件) 供运行时调用。当链接成功时会显示链接后形成的库文件 (.lib文件) 和 (.exp文件)，当链接错误时，结果视图内会显示出错的原因。该菜单项只在需编程实验中可用。

运行菜单项：显示运行界面，加载当前目录下的动态链接库文件 (.dll文件) 显示运行结果。该运行界面为Flash界面，您可在界面进行相应的操作。例如运行界面通用的“运行”“停止”“退出”等控制命令，一些涉及具体实验的控制动作，例如基础实验运行界面的点击话机、拨号等动作，具体的实验控制动作请参见相应的实验部分，这里不再赘述。

演示菜单项：提供正确的实验运行效果，供参考对照。演示界面和运行界面相同，但内部的控制逻辑均已预先确定，可在任何时候点击观看。

提示说明：我们的编译命令编译的是当前视图的源文件内容，因此会自动保存当前视图内容；链接命令会链接当前目录下的对象文件；运行命令会运行当前目录下的动态链接库文件，所以如果您对源文件进行了改动，想确定新源文件的逻辑正确性，请您依次进行编译、链接以产生新的dll文件。

4.在“帮助”菜单下包括了“实验手册”和“有关我们”的菜单项。

实验手册：包括了实验平台的使用说明、实验流程以及各个实验的相关原理、编程实验需要的的数据结构和运行时的情况分析。

有关我们：有关本实验系统的版本和版权等信息。

四.有关实验平台的工具栏和状态栏：

工具栏：本实验系统为了便于您操作，提供了一些常用的工具栏，固定安排在编辑视图之上，从左到右的工具按钮为：保存、撤销、剪切、复制、粘贴；编译、链接、运行、演示和帮助，您把鼠标停放在某工具按钮上，半秒以后会显示该工具按钮的用处。

状态栏：安排在结果视图以下，当您把鼠标指向某菜单命令上时，状态栏中会显示该菜单命令的描述信息；用于指示键盘“Num Lock”、“Caps Lock”、“Scroll Lock”灯亮灭的指示信息安排在状态栏的右侧。

五.有关实验系统的目录设置：

在您的安装目录下的目录和文件说明如下：

1.SourceFile 目录：保存了需编程实验需要的源代码文件；

2.Scfile_BackUp目录：保存了需编程实验需要的源代码文件的备份，该目录下的文件请不要改动；

3.Utility目录：包括了编程实验编译时需要的工具和需要的库文件、头文件等；实验运行演示时需要的音信号和动画文件也存放在这里；该目录下的文件请不要改动，否则可能会给您的实验带来不利影响；

4.BackUp目录：实验系统的备份文件；

5.现代交换原理仿真实验系统.exe：本实验系统的应用程序；

6.uninstall.exe：本实验系统的卸载程序，你也可以通过控制面板的“添加\删除程序”工具进行删除；其它部分为本实验系统安装时的一些说明和安装信息等，为了便于您的使用，我们在你的桌面和开始菜单中也设置了您可能需要的快捷方式。

交换仿真实验实验流程

交换仿真实验系统中包括两类实验：需编程实验和非编程实验。

1. 点击“现代交换原理仿真实验系统.exe”进入本实验系统，点击主界面“帮助”菜单下的“帮助手册”打开交换仿真实验帮助系统，阅读《交换仿真实验平台简介》部分，熟悉实验环境。

2. 点击主界面的“选择实验”菜单下的某个实验，进入该实验部分，将看到界面给你的提示。

2.1 对于需编程实验：

2.1.1 阅读帮助系统中有关该实验的内容，包括实验原理、实验所编函数的用处和该函数的输入输出参数、实验所编函数的流程图等；

2.1.2 在主界面的编辑视图内完成源代码部分的编写工作；

2.1.3 点击“编译”菜单命令或者“编译”工具按钮编译源代码，如果在结果视图中看到形成对象文件的提示（即产生***.obj），表示编译成功，进行2.1.4，否则返回2.1.2，根据结果视图的错误信息修改源程序；

2.1.4 点击“链接”菜单命令或者“链接”工具按钮链接产生的对象文件，如果在结果视图中看到形成目标模块文件（即产生***.lib和***.exp），表示链接成功，进行2.1.5，否则返回2.1.2，修改源代码；

2.1.5 点击“演示”菜单命令或者“演示”工具按钮，进入演示运行界面，进行相应的控制操作，观看正确的运行结果。

2.1.6 点击“运行”菜单命令或者“运行”工具按钮，进入演示运行界面，进行相应的控制操作，检验完成的目标模块的逻辑正确性。如果正确的话，点击“编辑源文件”下

的“保存”子菜单项保存该源文件。否则返回2.1.2修改源代码。

验收标准：在验收人员的监督下，完成编译、链接工作，进行运行确定代码逻辑的正确性。

注意：如需完成实验报告，在退出实验系统前点击“编辑源代码”下的“另存为”子菜单”，推荐把源文件保存在其他的目录下。**不要把源文件保留在本实验系统的安装目录以及其子目录下，否则该源文件可能会在下次实验系统启动时被改写。**

2.2 对于非编程实验：

2.2.1 阅读帮助系统中有关该实验的内容，包括实验原理、在实验运行界面的操作以及以及正确时的实验现象。

2.2.2 按照主界面对话框的提示，直接点击“编译运行”菜单下的“运行”子菜单，或者点击工具栏上的“运行”按钮，进行相应实验。

验收标准：在验收人员的监督下，在运行界面完成所需表格的填写，进行运行确定代码逻辑的正确性。

3. 如还需进行其他实验，点击主界面的“选择实验”菜单下的其它实验项，进入相应实验部分。

基础实验部分概述

链接：[基础实验总体设计](#)

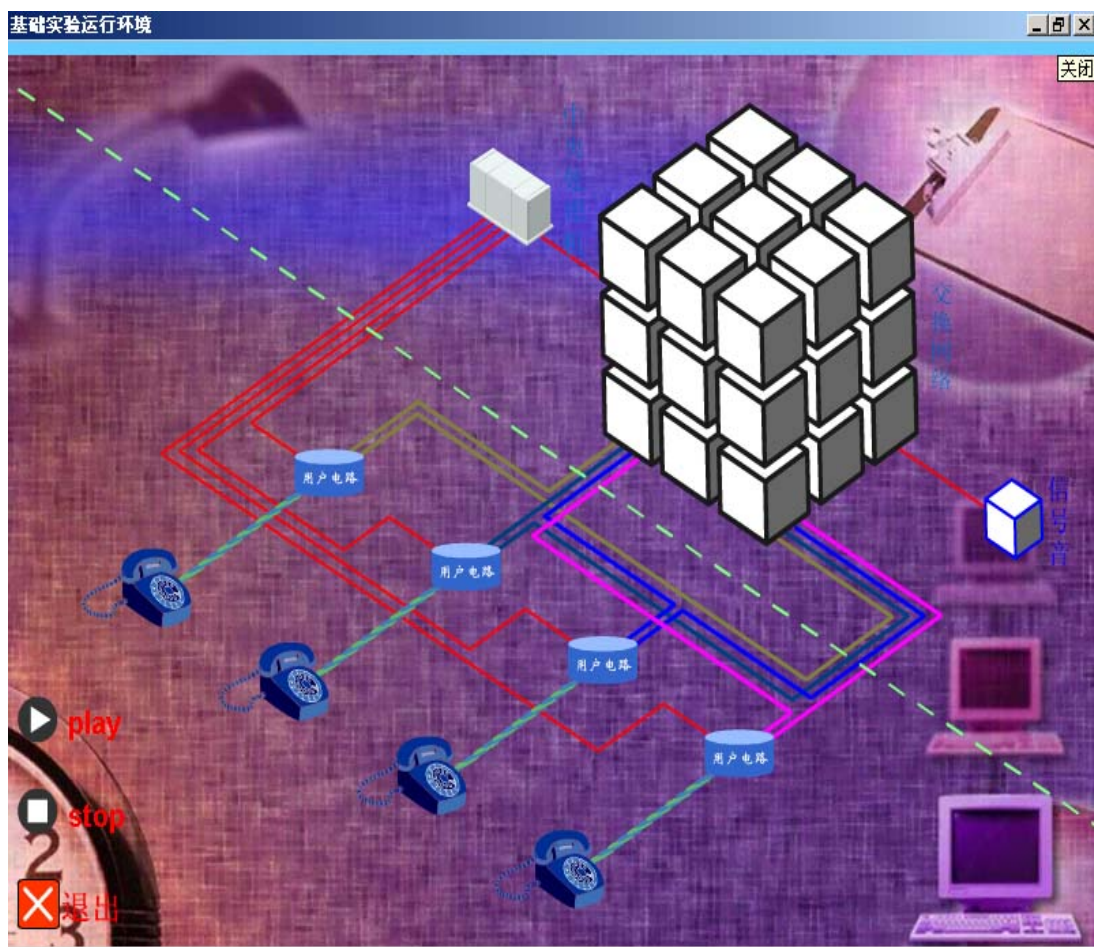
链接：[基础实验实验设置](#)

一.基础实验总体设计：

基础实验部分主要是加深学生对程控交换机中电路交换方式的理解，向学生提供一台模拟的“程控数字交换机”，该交换系统是一个具有四个话机终端的本局交换系统，能够进行呼叫处理，并且具有发送各种信号音、振铃信号以及通话接续等功能。

这里所谓的“模拟的”是指交换机的所有硬件动作均由软件模拟来完成。在基础运行（演示）界面（如下所示），学生使用鼠标点击话机来模拟摘机、挂机动作，使用鼠标点击号码盘的相应数字来模拟拨号动作，通过程序的内部控制能够驱动界面flash显示：例如摘机情况下能够听到拨号音、弹出拨号号盘；在接通被叫时被叫方话机振铃、主叫方听到回铃音；被叫方摘机时，能够显示网络接通等。

下面是基础实验的演示运行环境，作为对上面描述的补充，具体的界面操作我们将在《基础实验运行演示环境》一节中介绍。



二.基础实验实验设置：

在基础实验部分，我们预设了六个实验，均为需编程实验，要求学生完成指定的一些逻辑功能。

时间表调度实验：考查学生对时间表调度原理的掌握情况；

摘挂机检测实验：考查学生对摘挂机检测原理的掌握情况；

脉冲计数实验：考查学生对脉冲识别原理的掌握情况；

位间隔识别实验：考查学生对位间隔识别原理的掌握情况；

软件送音实验：考查学生对各种信号音发送情况的掌握情况；

驱动交换网络实验：考查学生对T接线器时隙交换原理的掌握情况。

基础实验一 时间表调度实验

链接：[实验目的](#)

链接：[实验原理及设计](#)

链接：[实验主要数据结构](#)

链接：[实验效果检验](#)

一.实验目的：

驱动交换网络实验用来考查学生对时间表调度原理的掌握情况。

二.实验原理及设计：

在程控数字交换的体系结构中，周期级程序（例如摘挂机检测程序、脉冲识别程序、位间隔识别程序）是由时间表调度实现的。所谓时间表调度，是指每经过交换系统的最短有效时间（这通常是指各周期性程序周期的最大公约数），都会检查调度表的调度要求，如果某个程序在这时需要执行，则调度程序开始执行它。

在我们设计的时间表调度实验中，这个调度表的调度是静态的。所谓静态，是指我们的调度表是在系统初始化的时候就建立起来的，在系统运行的情况下不再改动。实验要求的就是这个调度表的初始化。这个调度表如下：

我们这个交换系统提供了三个周期性调度程度（摘挂机检测程序、脉冲识别程序和位间隔识别程序），它们的调用周期分别为200ms、10ms和100ms，所以我们系统的最小调度时间为10ms。如图所示，每隔10ms,我们就会检查这个表的一行，如果该行上某一列为1，我们就执行所对应的任务，如果为0，就什么都不做。每当执行到这个表的最后一行，调度任务会返回第一行循环执行。而你所要做的就是按照你的理解来填写这个调度表。

三.实验主要数据结构：

函数功能：完成调度表的初始化；

函数原型：initSchTable(int ScheduleTable[SchTabLen][SchTabWdh])；

其中SchTabLen和SchTabWdh为在bconstant.h中的宏定义：

```
#define SchTabLen 20 //代表这个调度表为20行（相邻行之间的时间间隔为10ms);
```

```
#define SchTabWdh 3 //代表三个周期性调度任务——  
0：摘挂机检测任务；1：脉冲检测任务；2：位间隔检测任务；
```

四.实验效果检验：

当调度表初始化正确时，能够进行正常的通话；如果初始化不正确，可能会造成周期性程序的不正常调用，例如位间隔调度的延迟会造成识别位间隔的延误甚至丢失。

注：由于为循环程序，所以调度表的初始化方案不唯一。

基础实验二 摘挂机检测实验

链接：[实验目的](#)

链接：[实验原理及设计](#)

链接：[实验主要数据结构](#)

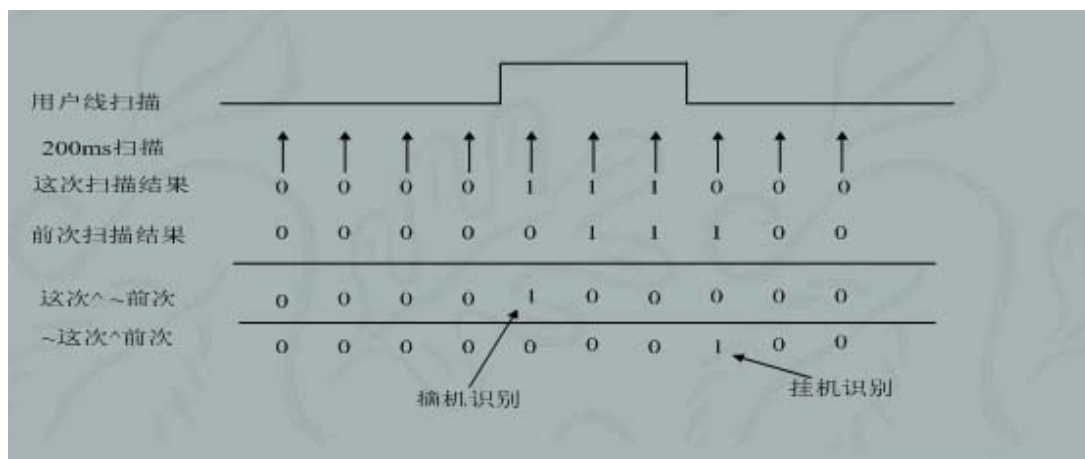
链接：[实验主体流程图](#)

一.实验目的：

摘挂机检测实验用来考查学生对摘挂机检测原理的掌握情况。

二.实验原理及设计：

设用户在挂机状态时扫描输出为“0”，用户在摘机状态时扫描输出为“1”，摘挂机扫描程序的执行周期为200ms，那么摘机识别，就是在200ms的周期性扫描中找到从“0”到“1”的变化点，挂机识别就是在200ms的周期性扫描中找到从“1”到“0”的变化点，该原理的示意图如下所示：



在我们的实验中，我们把前200ms的线路状态保存以备这次可以读取，同时读出这次的线路状态，把前200ms的线路状态取反与这次的线路状态相与，如果为1，就说明检测到摘机消息了。同理，我们把这次的线路状态取反再与前200ms的线路状态相与，如果为1就说明检测到挂机消息了，然后把摘挂机信号作为事件放入摘挂机队列中。

三.实验主要数据结构：

函数功能为：检测到摘、挂机事件，并把该事件放入到摘挂机事件队列中。

函数原型：void scanfor200(int linestate200[LINEMAX],int linestate[LINEMAX],UpOnnode * head1, UpOnnode* end1)；其中LINEMAX为线路总数，是定义在”bconstant.h”中的一个宏,linestate200[LINEMAX]为已保存的200ms前线路状态，linestate[LINEMAX]为当前的线路状态，head1,end1为摘挂机队列的首尾指针，该队列已经在主程序中进行了初始化。我们所要做的就是检测到的摘挂机事件以摘挂机队列节点的形式插入到摘挂机事件队列中。

数据结构说明：

头文件：“bconstant.h”;(以下的数据结构都已在该文件中定义)

LINEMAX：最大线路数；

int linestate200[LINEMAX],linestate[LINEMAX]：线路从0开始编号；状态：1：有电流，0无电流；

enum UporOn {ehandup,ehandon}：为摘挂机区别符；ehandup表示摘机，ehandon表示挂机；

```
struct UpOnnode    //摘挂机队列节点结构    {
    UporOn phonestate;    //摘挂机区别符；
    int linenum;          //线路号（从0开始）；
    struct UpOnnode* next; //指向下一节点的指针；
};
```

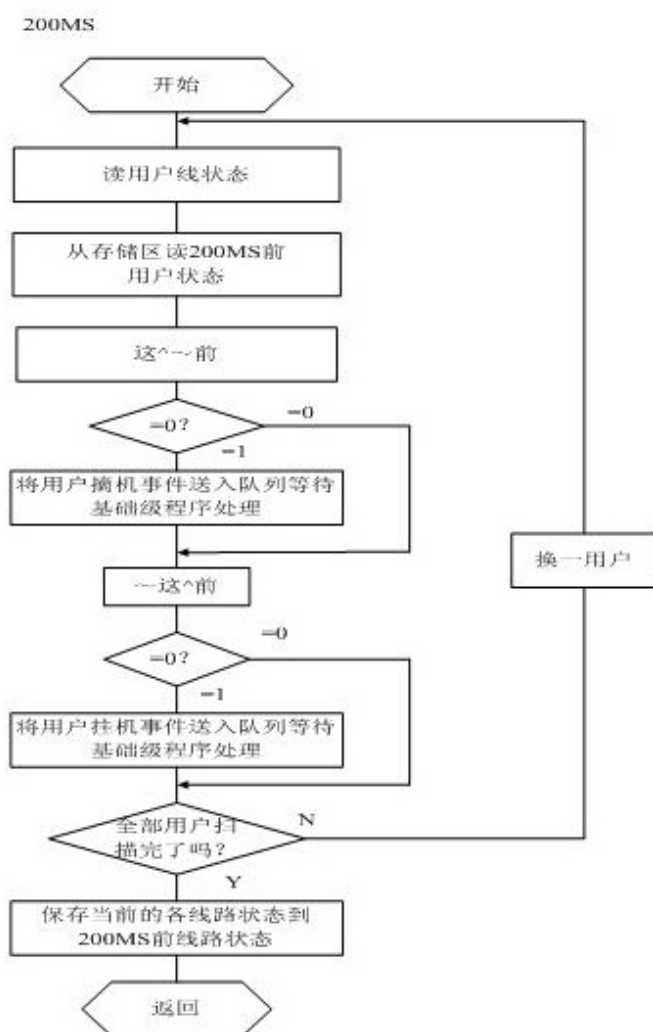
注意事项：

1.我们编写的模块是基础实验部分预加载的本局交换系统的一个模块而已，在系统中head1头指针和end1尾指针已经完成初始化。为方便起见，我们的摘挂机事件队列是一个包含头节点的单向链表，并且头指针指向该头节点，尾指针在初始化时也指向了该节点。所以在我们的函数编写中

应保证头指针始终指向 该头节点上、尾指针指向摘挂机事件队列的最末一个节点。

2.注意把这次扫描的线路状态值保存在前200ms扫描线路状态数组中,以便主程周期调用。

四.实验主体流程图：



基础实验三 脉冲计数实验

链接：[实验目的](#)

链接：[实验原理以及实验设计](#)

链接：[实验主要数据结构](#)

链接：[实验主体流程图](#)

一.实验目的：

脉冲计数实验用来考察学生对脉冲识别原理的掌握情况。

二.实验原理以及实验设计：

拨号盘所发出的拨号脉冲有规定的参数。我国规定的号盘脉冲的参数有：

脉冲速度：即每秒钟送出的脉冲个数，规定的脉冲速度为每秒钟8-16个脉冲；

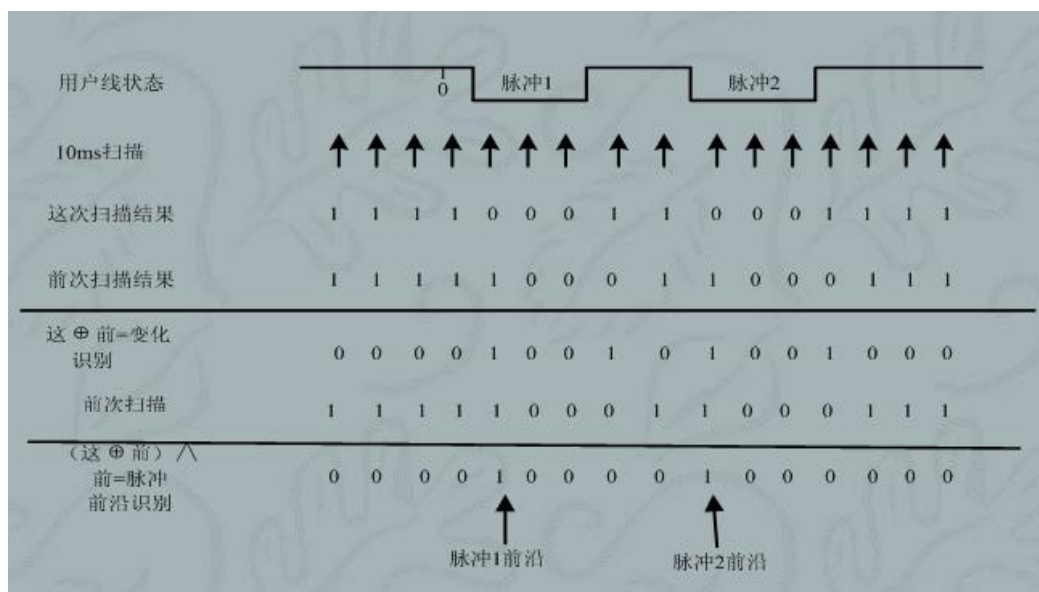
脉冲断续比：即脉冲宽度（断）和间隔宽度（续）之比，规定的脉冲断续比为1：1-3：1。

1) 脉冲识别程序扫描周期的确定：

为确定脉冲识别扫描的周期，需要计算出最短的变化间隔（脉冲或间隔宽度），这样才能保证每个脉冲都能够识别而不至于丢失脉冲。由于号盘每秒发出的最快脉冲个数为16个，脉冲周期 $T=1000/16=62.5\text{ms}$ ，在这种情况下断续时间比为3：1时续的时间最短，为 $1/4*T$ ，所以最短变化周期为 $1/4*(1000/16)=15.625\text{ms}$ ，脉冲识别扫描程序的周期 $<15.625\text{ms}$ 。

2) 拨号脉冲识别原理：

在下图中，采用了10ms的扫描周期，其中的变化识别标志了状态的变化。对于一个脉冲来说，是前沿和后沿各识别一次，我们可以任取一个来识别脉冲，下图中采用了前沿识别。从逻辑上讲，也就是说（这 $\oplus$ 前） $\wedge$ 前 $=!$ 这 $\wedge$ 前相当于前面所说的挂机识别，同样（这 $\oplus$ 前） $\wedge!$ 前 $=!$ 这 $\wedge!$ 前相当于摘机识别。在这里采用比较麻烦的逻辑运算的原因是需要“变化识别”这个结果。这在位间隔识别中要用到，下面是脉冲识别原理示意图：



在我们设计的实验中，用一个数组保留各线路10ms前的状态，用另一个数组保留各线路当前的状态，并且提供了保存“变化识别”的数组（以供后面的位间隔识别使用），另外提供给学生使用的是保存已检测的脉冲值的数组，学生编程检测到一个脉冲以后，就将该线路对应的脉冲值加一。

三.实验主要数据结构：

函数功能：识别出一个脉冲，然后把该线路对应的脉冲数加一。

函数原型：void scanpulse(int linestate[LINEMAX],int linestate10[LINEMAX],int change [LINEMAX],int fchange[LINEMAX],int pulsenum[LINEMAX]) 其中LINEMAX为最大线路数，linestate为当前线路状态，linestate10为10ms前的线路状态，change为状态改变，fchange为首次变化，pulsenum为脉冲计数数组。

主要的数据结构：

头文件：“bconstant.h”;(以下的数据结构都已在该文件中定义)

LINEMAX：最大线路数；

int linestate[LINEMAX],linestate10[LINEMAX]：线路从0开始编号；状态：1：有电流，0无电流；

int change[LINEMAX]：线路状态变化数组，0表示无变化，1表示有变化；

int fchange[LINEMAX]：线路首次变化数组，1表示已发生首次变化，0表示没有；

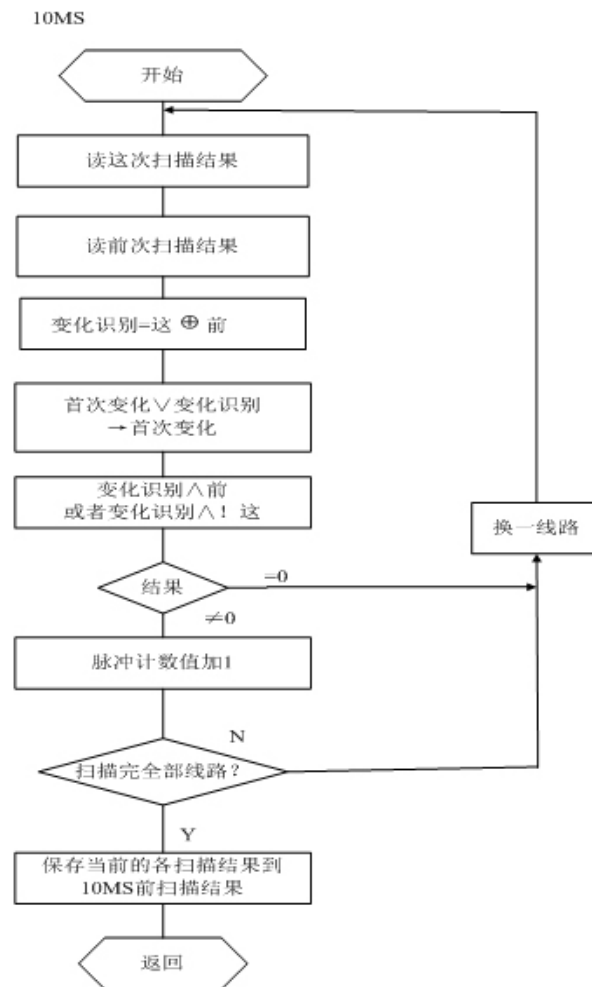
int pulsenum[LINEMAX]：线路当前的脉冲值；

为方面大家操作，我们提供了两个预定义的函数：

`int nor_op(int a,int b);int or_op(int a,int b);`分别用于异或操作和或操作。

注意事项：把这次的线路状态值保存到10ms前的线路状态数组中，以便主程序周期调用。

四.实验主体流程图：



基础实验四 位间隔识别实验

链接：[实验目的](#)

链接：[实验原理及设计](#)

链接：[实验主要数据结构](#)

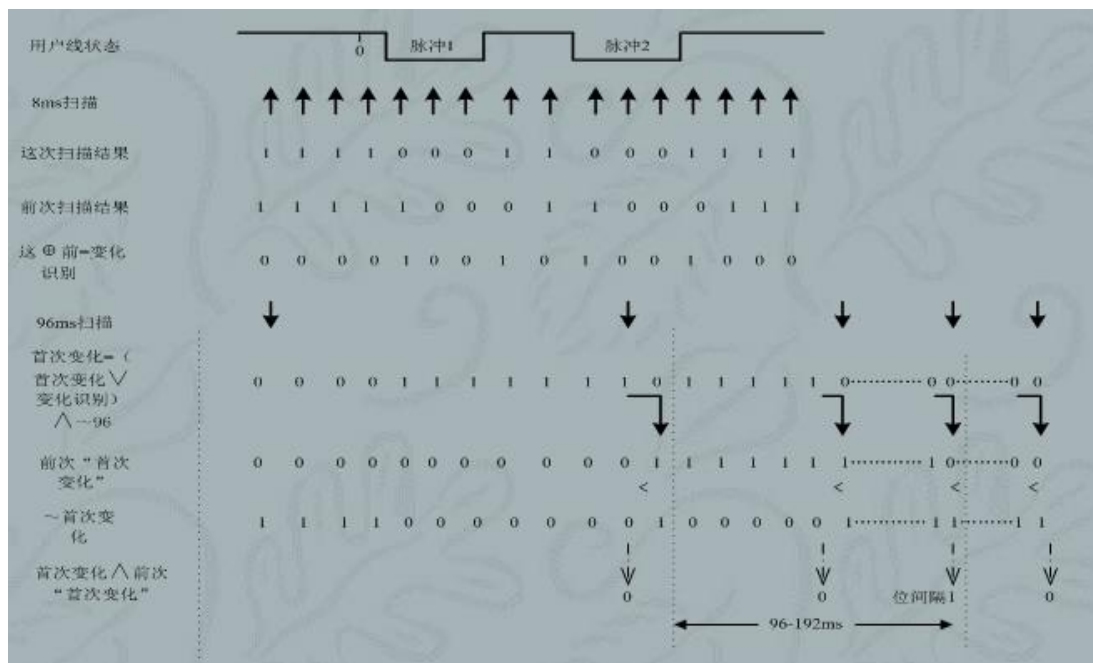
链接：[实验主体流程图](#)

一.实验目的：

位间隔识别实验用来考察学生对位间隔识别原理的掌握情况。

二.实验原理及设计：

位间隔识别的目的是要识别两位号码之间的间隔，从而区分各位号码。首先来确定一下位间隔识别的扫描周期，一方面拨号盘的位间隔 $\geq 250\text{ms}$ ，另一方面需要确定一下最长的脉冲或者间隔为多少毫秒。由于最慢的脉冲速度为每秒8个脉冲，这就是说脉冲周期 $T=1000/8=125\text{ms}$ ，当断续比为3：1时，脉冲（断）时间应为 $125\text{ms} \times 3/4 = 93.75\text{ms}$ ，所以位间隔识别程序要能鉴别93.75ms和250ms间的间隔。一般采用96ms扫描程序来识别，位间隔识别的原理图：



对于位间隔识别的基本原理，在这里要强调两个关键点

(1) 识别在前96ms周期内没有发生过变化。这就排除了脉冲变化的因素。因为脉冲最长间隔如前面所计算的那样为93.75ms < 96ms；

(2) 识别出在此以前的最后一次变化是在96ms以前的那个周期内，这一条件可以保证在位间隔开始96ms后的第一个周期就能识别到，而且保证以后各次扫描不识别。

为此引入了“首次变化”这个变量，它标志首次遇到了“变化”。平时它为“0”，当在一个扫描周期内遇到变化后就变为“1”，在这个周期的后续时间里它都保持“1”不变。这个条件可由下面的逻辑关系来实现：首次变化=首次变化 $\vee$ 变化识别。当首次变化为“0”时，只要8ms脉冲扫描的变化识别为“0”，则首次变化永远为“0”，一旦变化识别为“1”，则首次变化就变为“1”，而且以后无论变化识别如何改变都不能改变首次变化的“1”值。为确保以上“首次变化”平时为“0”，令每次96ms程序都把它清“0”，这样就写出下面的计算公式：首次变化=(首次变化 $\vee$ 变化识别) $\wedge$ /96

在每次96ms程序执行期间来检查“首次变化”这个变量，若为“0”，说明在前96周期内没有发生过变化；若为“1”，说明已发生变化，但这时还不能确定为何种变化，既可能为脉冲变化，也可能为位间隔变化。这就需要看下一个96ms周期，若仍有变化，则属于“脉冲变化”；若无变化，即属于“位间隔”变化(>96ms无变化)。这时在再下一个周期内仍能识别出“无变化”，但已经识别出一次了，不能再作重复识别。

对上面的讨论加以概括，只要识别两个变量就可以了：(1) 上一个96ms周期内无变化；(2) 再上一个周期内有变化就可确定为“位间隔”了。在上面的图中的“首次变化”是识别变量(1)的，首次变化=1是说明上一周期内无变化，否则有变化。上图中前次“首次变化”是识别后一个变量(2)的，前次“首次变化”是读取“首次变化”的存储内容，不过96ms读一次，读的正是再上一个周期的最后结果。前次“首次变化”=1，说明再上一个周期有过变化，否则无变化。将“首次变化”取反后与前次“首次变化”相与，结果为“1”，表示有位间隔。

值得注意的是，仅上面识别出的“位间隔”还是不够的，因为它只能说明前一次变化在96ms以前，那么用户中途挂机也可以达到这个条件，因此必须区别是“位间隔”还是“中途挂机”。区别这个很容易，只要区别一下现在用户处于挂机还是摘机状态就可以了。前者是中途挂机，后者是位间隔。方法可以是查一下当前的用户线状态，如果为“1”，说明用户已经挂机，那么识别的是“中途挂机”，否则为“位间隔”。

在我们的实验中提供了前100ms的线路状态数组和当前的线路状态数组，以及用来保存“变化识别”、“首次变化”和“前次首次变化”的数组等供学生使用，学生通过编程检测位间隔的位置，当学生检测到位间隔以后，学生把与该线路对应的脉冲值放入到一个接收号码队列中，并且放在接收号码队列中的事件需保存着与该号码对应的线路号，因为几个线路接收到的号码都是放在同一个队列中的。（然后，一个外部程序将扫描该接收号码队列，逐一把各个接收号码事件发送给交换系统中央控制系统。学生只要做到将号码事件按照接收号码队列节点格式放入队列即可。）

三.实验主要数据结构：

函数功能：识别出一位号码，将该号码放入到号码接收队列中；

函数原型为：void scandigit(int linestate[LINEMAX],int linestate100 [LINEMAX],int pulsenum[LINEMAX],int fchange[LINEMAX],int lfchange [LINEMAX], Digitnode * head2, Digitnode * end2)。其中LINEMAX为线路总数，是定义在bconstant.h中的一个宏，linestate[LINEMAX]为当前的线路状态，linestate100[LINEMAX]为100ms前的线路状态，pulsenum为当前已经检测到的脉冲计数值，fchange为首次变化，lfchange为前次首次变化，head2,end2为号码接收队列的首尾指针，该队列已经完成了初始化。本次实验就是把检测到位间隔的线路的号码值插入这个队列中。

数据结构说明：

头文件：“bconstant.h”；(以下的数据结构都已在该文件中定义)

LINEMAX：最大线路数；

int linestate[LINEMAX],linestate100[LINEMAX]：线路从0开始编号；状态：1：有电流，0无电流；

int pulsenum[LINEMAX]：线路已检测到的脉冲值数；

int fchange[LINEMAX],lfchange [LINEMAX]：首次变化数组，前次首次变化数组，有变化为1。

```
struct Digitnode          //号码接收队列节点数据结构
{
    int num;               //号码值
    int linenum;           //线号
```

```

struct Digitnode * next;//指向下一节点的指针
}

```

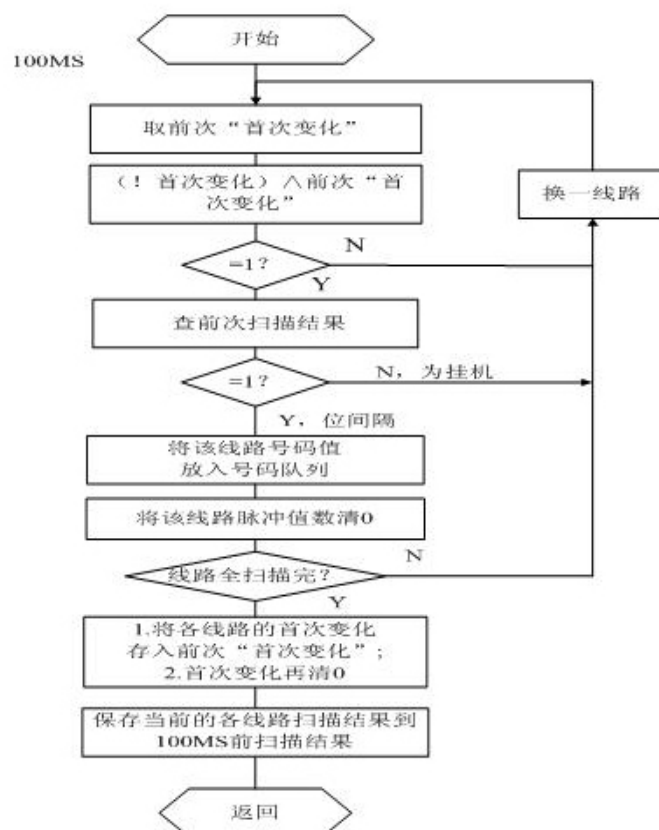
注意事项：

1.我们编写的模块是基础实验部分预加载的本局交换系统的一个模块而已，在系统中head2头指针和end2尾指针 已经完成初始化。为方便起见，我们的号码接收队列是一个包含头节点的单向链表，并且头指针指向该头节点，尾指针在初始化时也指向了该节点。所以在我们的函数编写中应保证头指针始终指向 该头节点上、尾指针指向号码接收队列的最末一个节点。

2.当某条线路检测到位间隔以后，该线路的脉冲值数应清0；在函数末尾应将当前线路状态保存在100ms前的线路状态中、首次变化保存在前首次变化中、首次变化再清0。

细心的您可能会问：“如果线路当前的脉冲数为10，需要把10做为号码值也插入吗”，的确，我们的号码值0是产生了10个脉冲，脉冲数组中脉冲数也会出现10，不过在您把脉冲值10插入到号码接收队列后，我们的后续程序会把它转化为0，传给交换控制系统的。

四.实验主体流程图：



基础实验五 软件送音实验

链接：[实验目的](#)

链接：[实验原理及设计](#)

链接：[实验主要数据结构](#)

链接：[实验检验标准](#)

一.实验目的：

软件送音实验用来考查学生对各种信号音发送情况的掌握情况。

二.实验原理及设计：

交换机需要向用户发送各种信号音，如拨号音、忙音和回铃音等，也需要向其他交换机发送和接收各种局间信令，如多频信号。这些信号都是音频模拟信号，而信号设备是接在数字交换网络上的，它通过数字交换网络所提供的路由来传送。因此这些模拟信号必须是“数字化了的”，信号音的产生不外乎单频音和双频音两种。对于单频信号音的产生，是按照一定的时间间隔抽样、量化和编码运算，得到各抽样点的pcm信号，然后保存在ROM中，在需要发送时按序发送。对于双频信号音的产生，是取得两种信号音周期的最小公倍数，作为pcm的抽样周期，然后保存在ROM中，在需要时按序取出发送。

由于用软件来模拟信号音的产生比较困难，所以在我们的实验中，我们发送的信号音都是预先准备好的信号音文件，并且伴随有界面的送音显示。那么如何考查学生对软件送音的理解呢？我的设计是提供给学生一些需要送音的情况，由学生编程确定该送音的类型，该送音的情况和送音的类别都是在主程序中预先安排好的，学生只需在送音情况和送音类型间“牵线搭桥”。

三.实验主要数据结构：

函数功能：根据输入的状态决定送音的类型；

函数原型：int decide(Userstate state)；其中state为输入的状态类型，输出为发送的信号音类型；

头文件：“bconstant.h”（以下的数据结构均定义在该头文件中）

```
enum Userstate {  
    caller_ehandup,      //主叫摘机  
    caller_calledbusy,   //被叫忙  
    theother_firsthandon, //通话中对方先挂机  
    dial_timer_timeout,  //拨号音超时  
    recvnum_timer_timeout, //收号定时器超时  
    connect_timer_timeout, //连接定时器超时  
    nulltone_timer_timeout, //空号音超时  
    ringback_timer_timeout, //回铃音定时器超时  
    busytone_timer_timeout, //忙音超时  
    caller_callednull,     //被叫空号  
    called_onecallin,      //（被叫）呼叫进入  
    caller_callconnected //（主叫）被叫接通  
};
```

注：有前缀caller的表示只有主叫才有的状态，called的表示被叫，其他为两者共同；

输出：

返回对该用户的送音类型：1：拨号音；2：振铃音；
3：回铃音；4：忙音；5：嚎鸣音；6：空号音；

四.实验检验标准：

为了配合界面Flash演示，运行时界面的Flash动画的信号音均为正确信号，您所决定的音信号将以声音方式播放，加以比较可以判断正误（您可在安装目录下的utility子目录下ringsound子目录中找到各种信号音文件）。

基础实验六 驱动交换网络实验

链接：[实验目的](#)

链接：[实验原理及设计](#)

链接：[实验主要数据结构](#)

链接：[实验效果检验](#)

一.实验目的：

驱动交换网络实验用来考查学生对T接线器时隙交换原理的掌握情况。

二.实验原理及设计：

在数字交换机中采用的是数字交换网络，数字交换网络的主要特点是它能够将从数字传输设备进来的数字信号直接进行交换，而不需要数/模和模/数转换。在数字交换中，常见的是T接线器和S接线器。由于基础实验中模拟的交换网络为T接线器，所以这里将只对T接线器进行简单的介绍。

T接线器有输出控制方式和输入控制方式两种，不管是哪一种方式，都由话音存储器和控制存储器两部分组成。在这里介绍一下输出控制方式，如果需要了解输入控制方式，请参阅有关书籍。

设输入话音信号在TS50上，要求经T接线器以后交换至TS450上去，然后输出到下一级。CPU根据这一要求，通过软件在控制存储器的450号单元写入“50”。这个写入是由CPU控制进行的，因此把它叫做“控制写入”。控制存储器的读出由定时脉冲控制，按照时隙号读出相应单元内容。话音存储器的工作方式正好和控制存储器的方式相反，即“顺序写入，控制读出”。也就是说，由定时脉冲控制，按顺序将不同的话音信号写入相应的单元中去。写入的单元号和时隙号一一对应。而读出时则要根据控制存储器的控制信息而进行。由于向话音存储器输入话音信号不受CPU控制，而输出话音信号受到由CPU控制的控制器的

控制，因此称为“输出控制”方式。在我们的例子中，语音的输入时隙号为50，在定时脉冲控制下就可写入到50号单元中。如前所述，CPU在控制存储器中的450号单元已写入了内容“50”。在定时脉冲控制下，在TS450这一时间，从控制存储器的地址450中读出内容为“50”，把它作为话音存储器读出地址，立即读出话音存储器的50号单元。这正好是原来在50号时隙写入的话音信号内容。因此在话音信号50号单元读出时已经是TS450了，即已把话音信号从TS50交换到TS450，实现了时隙交换。

在我们的驱动交换网络实验中，采用的是单T接线器。因为在这个模拟实验中，并没有任何的数据进行通信，所以在设计中并不存在话音存储器而只存在控制存储器。在建立交换网络时，要求学生在通信的两个时隙对应的控制存储器相应位置写入对方的时隙号，在拆除交换网络时，要求学生在通信的两个时隙对应的控制存储器相应位置清“0”。在显示交换网络建立或者拆除时，外部程序将会去查这个控制存储器来“建立”连接或者“拆除”连接，用这种方法就可以检测学生编码的正确性。

三.实验主要数据结构：

函数功能：完成通信双方通信时隙的确定和修改。

函数原型：`void connect_network(int timeslice_tnet[TIMESLICEMAX],int timeslice1,int timeslice2,int tag);`

其中 `TIMESLICEMAX` 为一个宏定义，定义在“bconstant.h”头文件中，表示系统可用的最大时隙数，`timeslice_tnet`为一个数组，代表了交换机内的一个控存（控制存储器），下标值（从0开始）相当于一个时隙索引，与下标值对应的数组值为与该时隙通话的时隙，`timeslice1`和`timeslice2`为欲建立或者拆除连接的两个时隙。`tag`为一个标志位，1表示建立网络，0表示拆除网络。

四.实验效果检验：

当建立连接成功时，在Flash界面可以看到通信双方“闪烁”的线路连接；

当拆除连接成功时，在Flash界面可以看到通信双方“闪烁”的线路消失。

基础实验运行演示环境

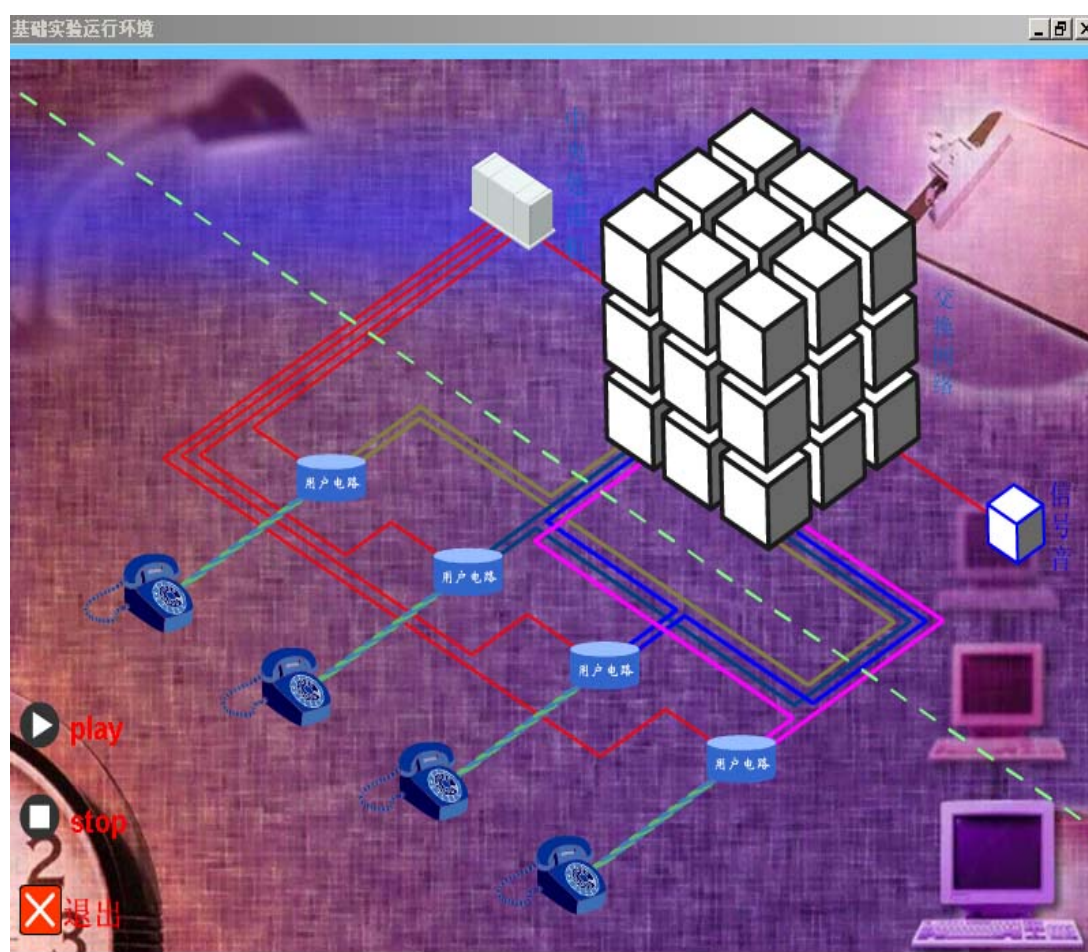
链接：[基础实验运行演示界面命令介绍](#)

链接：[一个通话的流程演示](#)

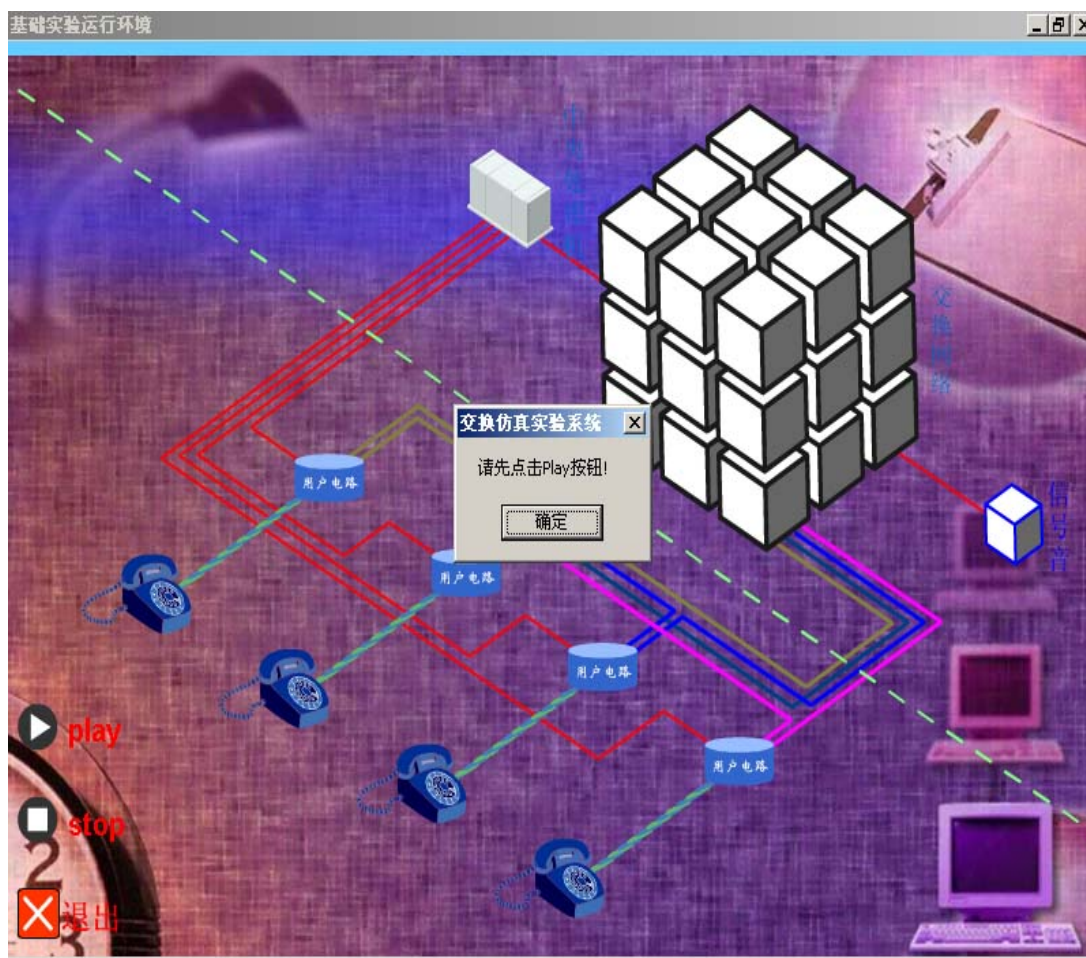
链接：[基础实验的错误分析](#)

一.基础实验运行演示界面命令介绍

下图是基础实验部分的运行演示界面，界面的主要命令有三个：“运行”、“停止”、“退出”。



“运行” (Play)命令：点击后使界面的电话机能够接受点击响应，如果不点击该命令按钮，就会出现：



“停止” (Stop)命令：点击该命令按钮，就会使界面的一切显示复原。这个命令很有用处，因为我们的线路状态变化是由软件程序模拟的，依赖于Windows系统提供的定时器，所以可能会出现号码“接收”错误，此时可以点击该命令使界面复原。

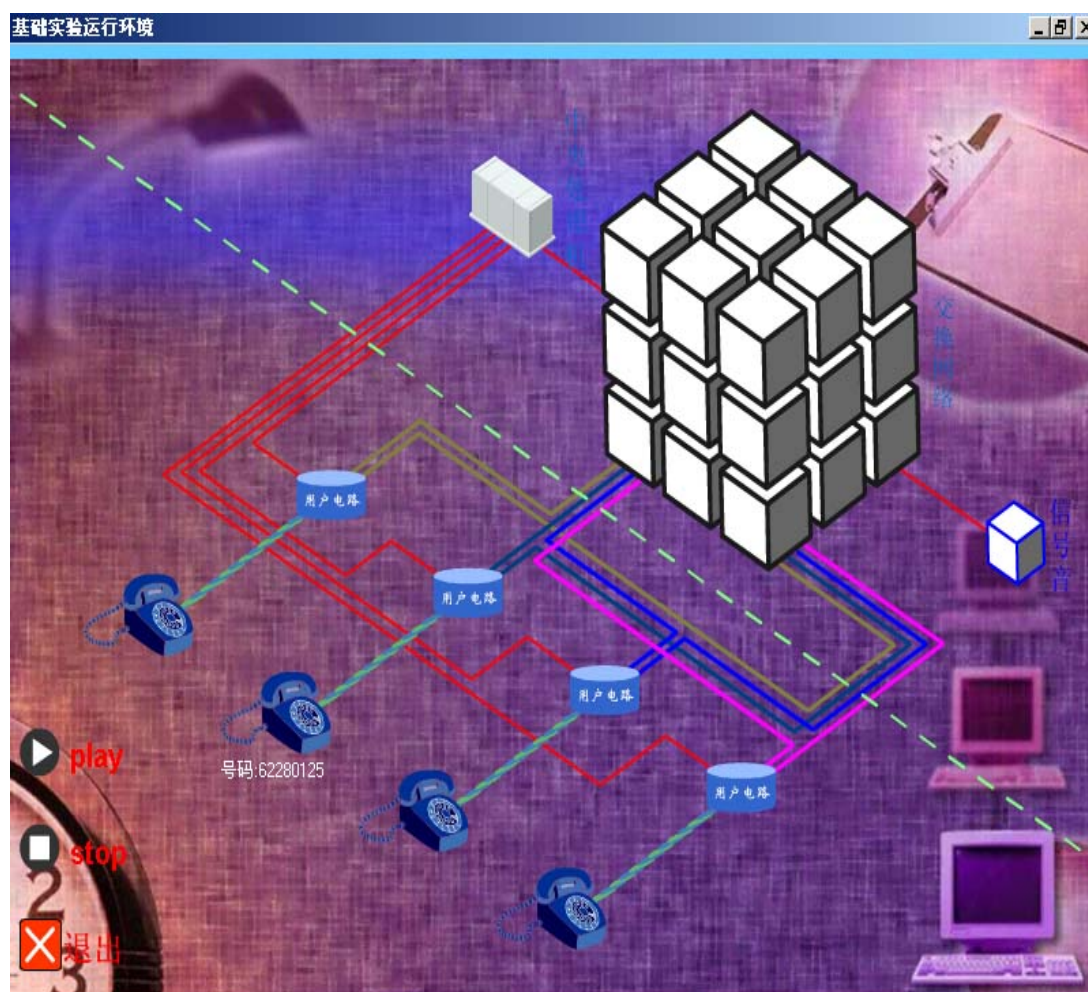
“退出”命令：点击该命令按钮，将退出该运行演示界面。

[下一页](#)

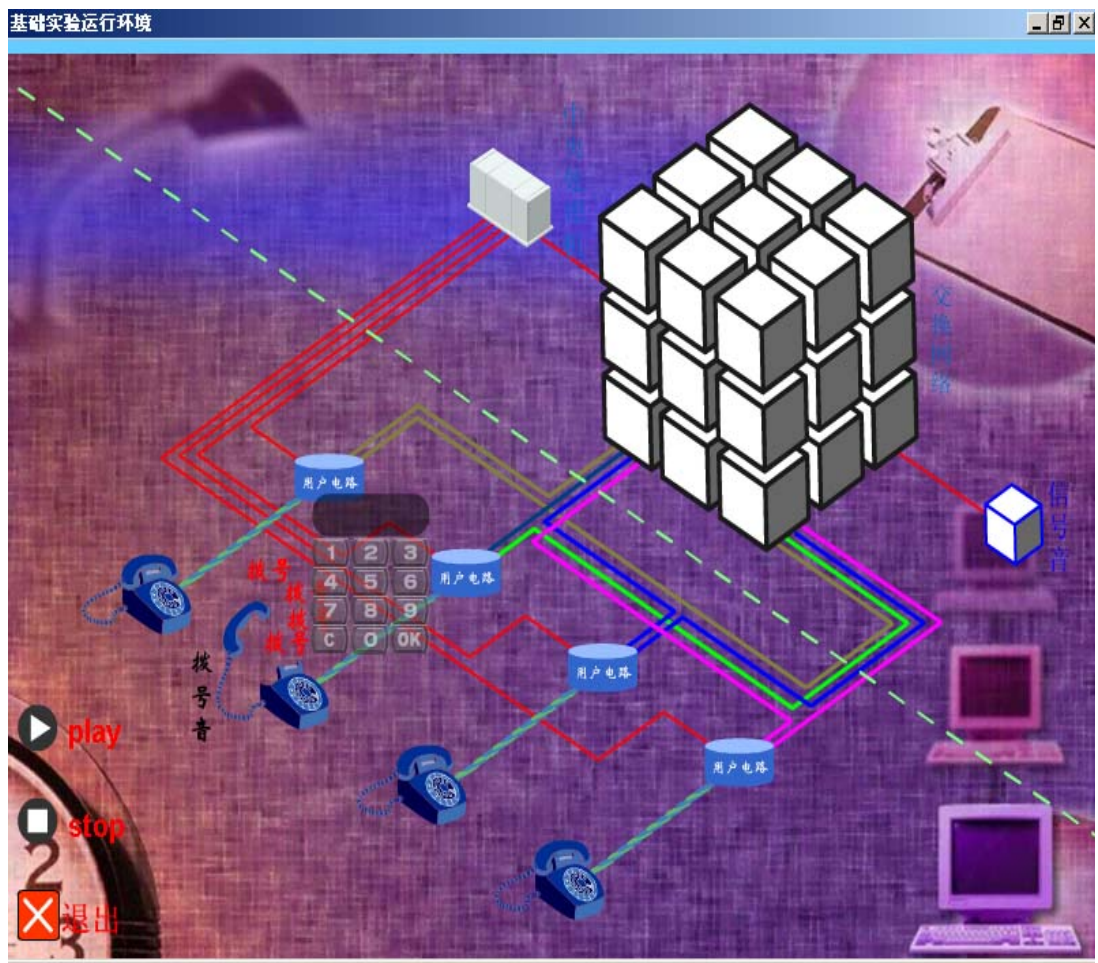
二.一个正常通话的流程演示：

下面，我们将演示一次通话的过程，以帮助大家熟悉该运行演示环境。

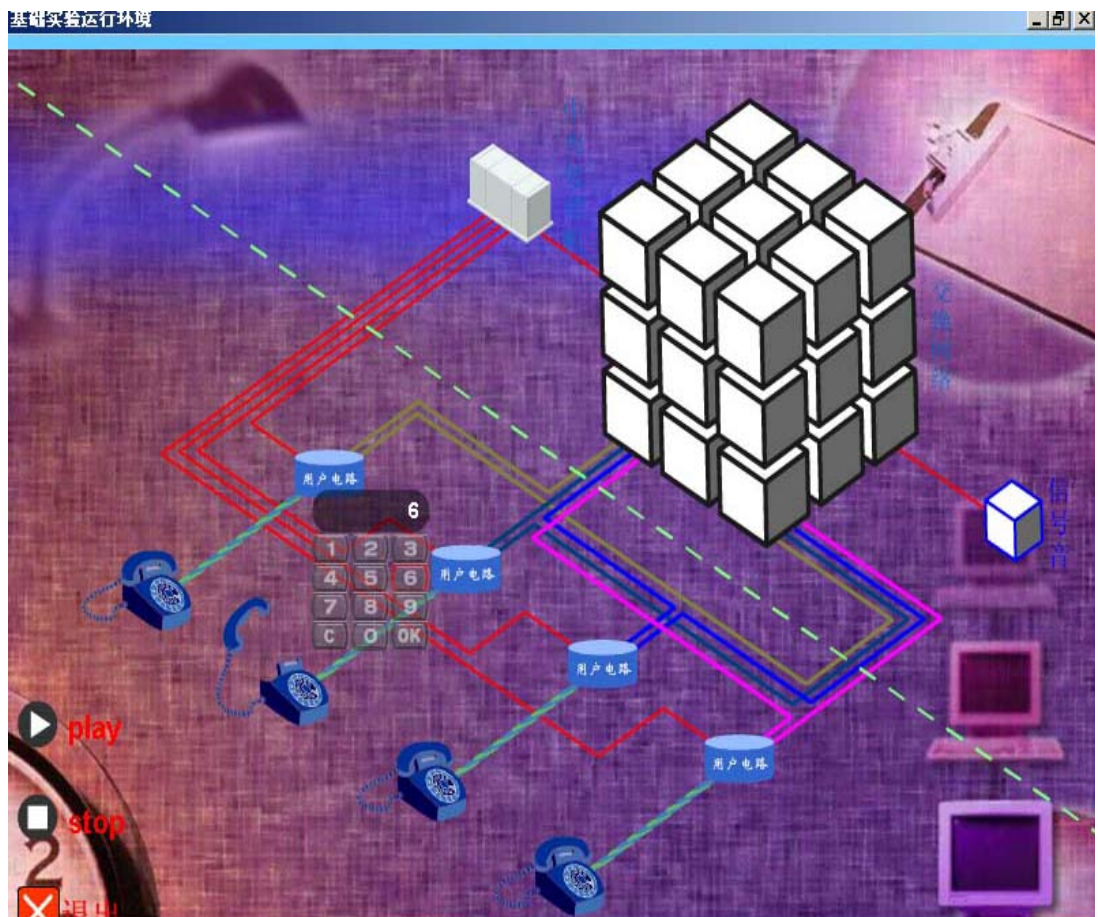
1.当我们把鼠标放在某电话机旁时，会显示该电话机的号码，我们的四台电话机的号码由上到下依次为：62280521、62280125、62280886、62283166。



2.我们选择一个话机作为主叫，例如62280125，点击该话机，将弹出号码盘，同时界面显示拨号音，我们在最初还可以听到拨号音。



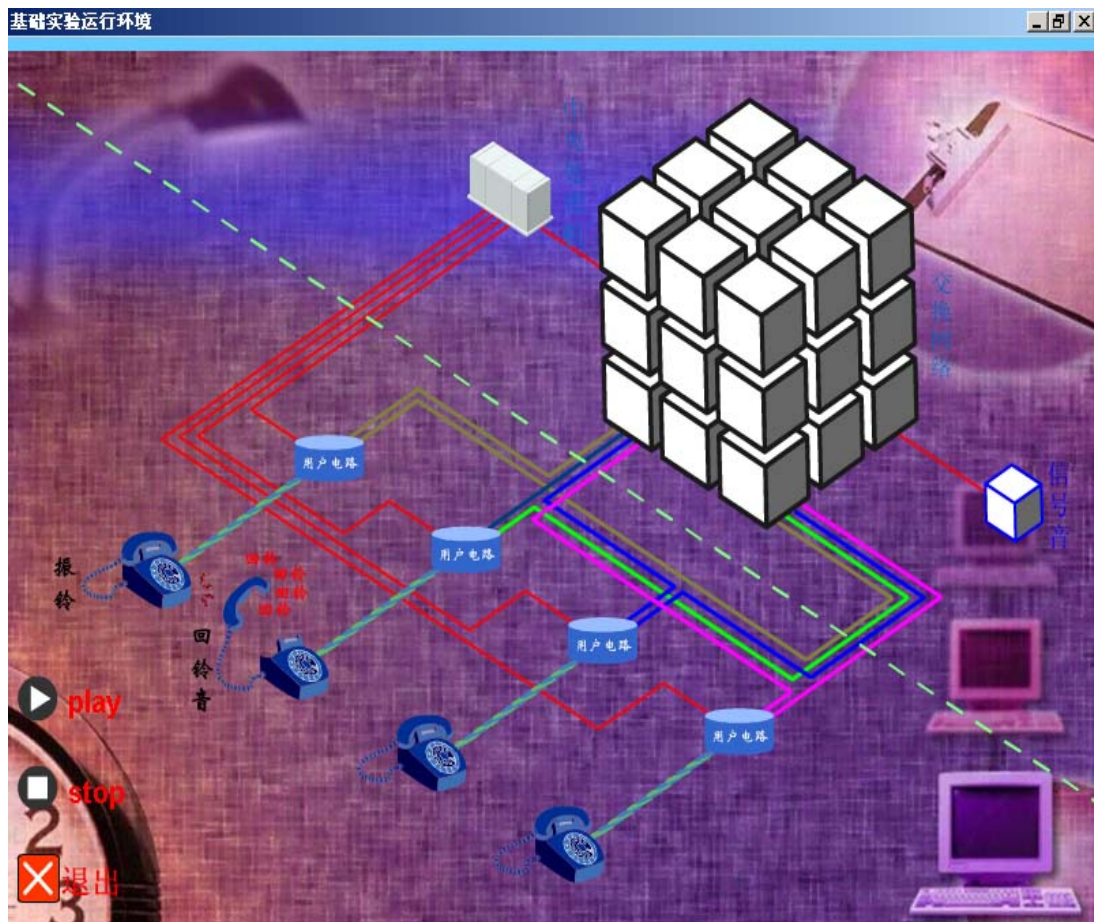
3. 这里我们不考虑异常情况，所以我们在拨号音超时前拨号，拨号音将停止，界面的拨号音显示也会消失。



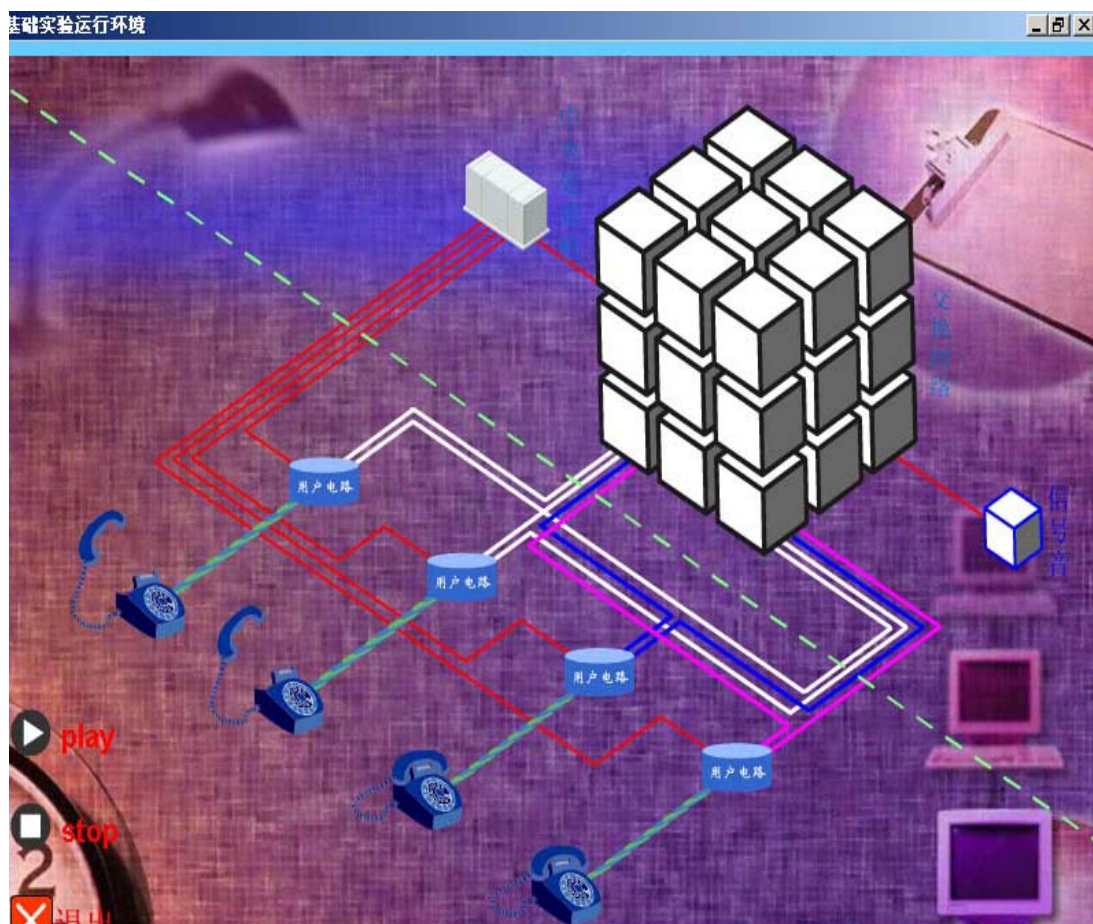
[上一页](#)

[下一页](#)

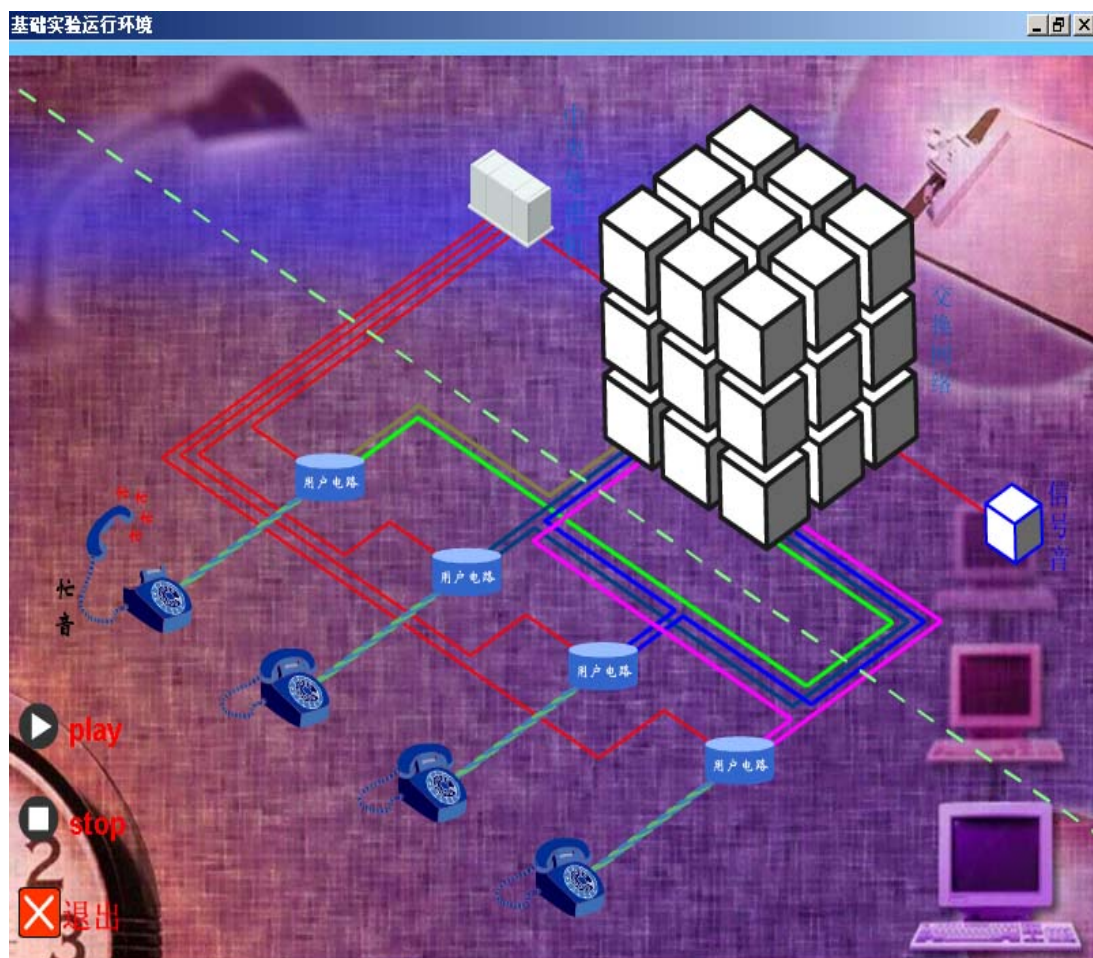
4.继续拨号，我们拨通62280521，这时界面将显示62280521的话机在振铃、我们的话机显示回铃音，同时我们在最初还可以听到振铃音。



5.我们点击被叫话机，表示被叫接听。这时，界面的振铃和回铃音显示均消失，进入通话状态，我们可以看到两个话机之间有线路连接，表示通话。



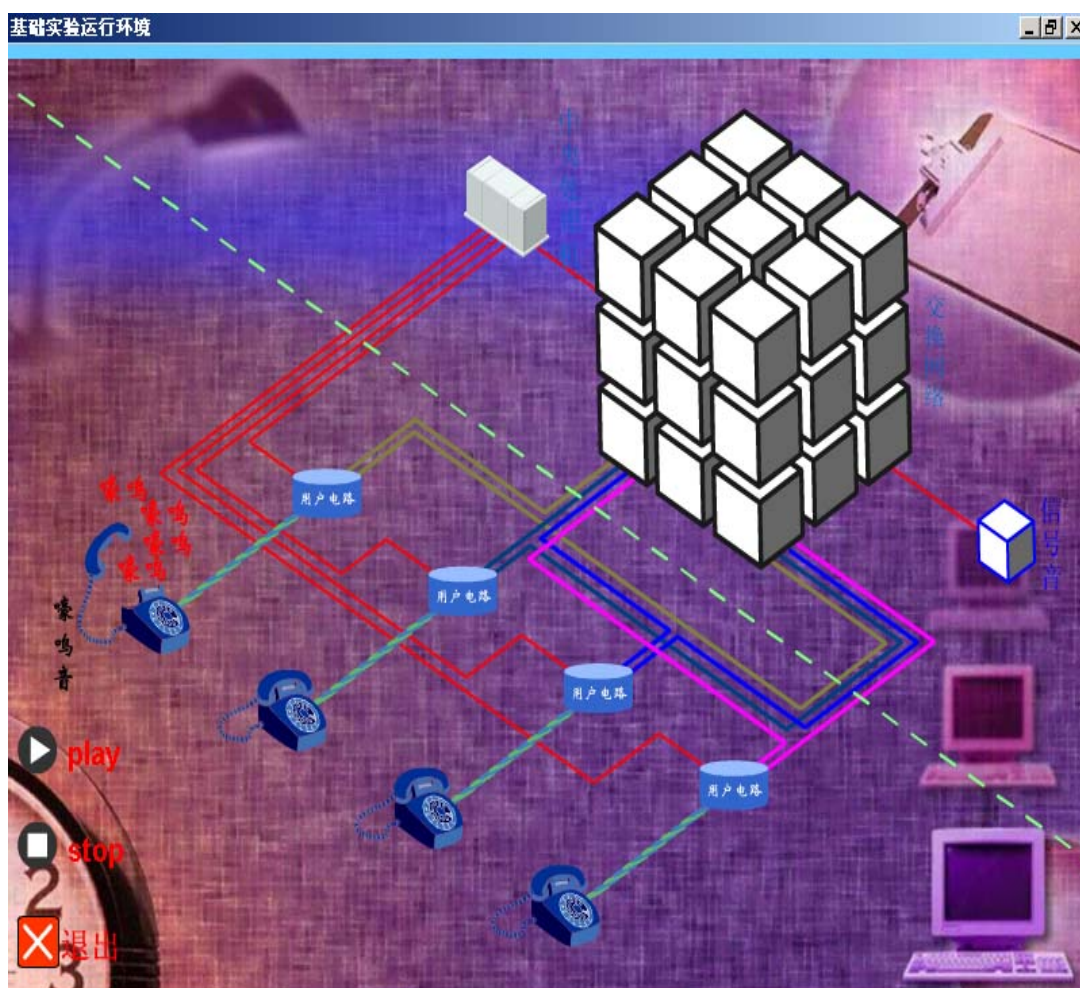
6.我们假设主叫通话后先挂机，则被叫将听到忙音，界面将出现忙音状态。（我们在拨号音超时、拨号位间超时、被叫忙的时候都会进入到忙音状态）。



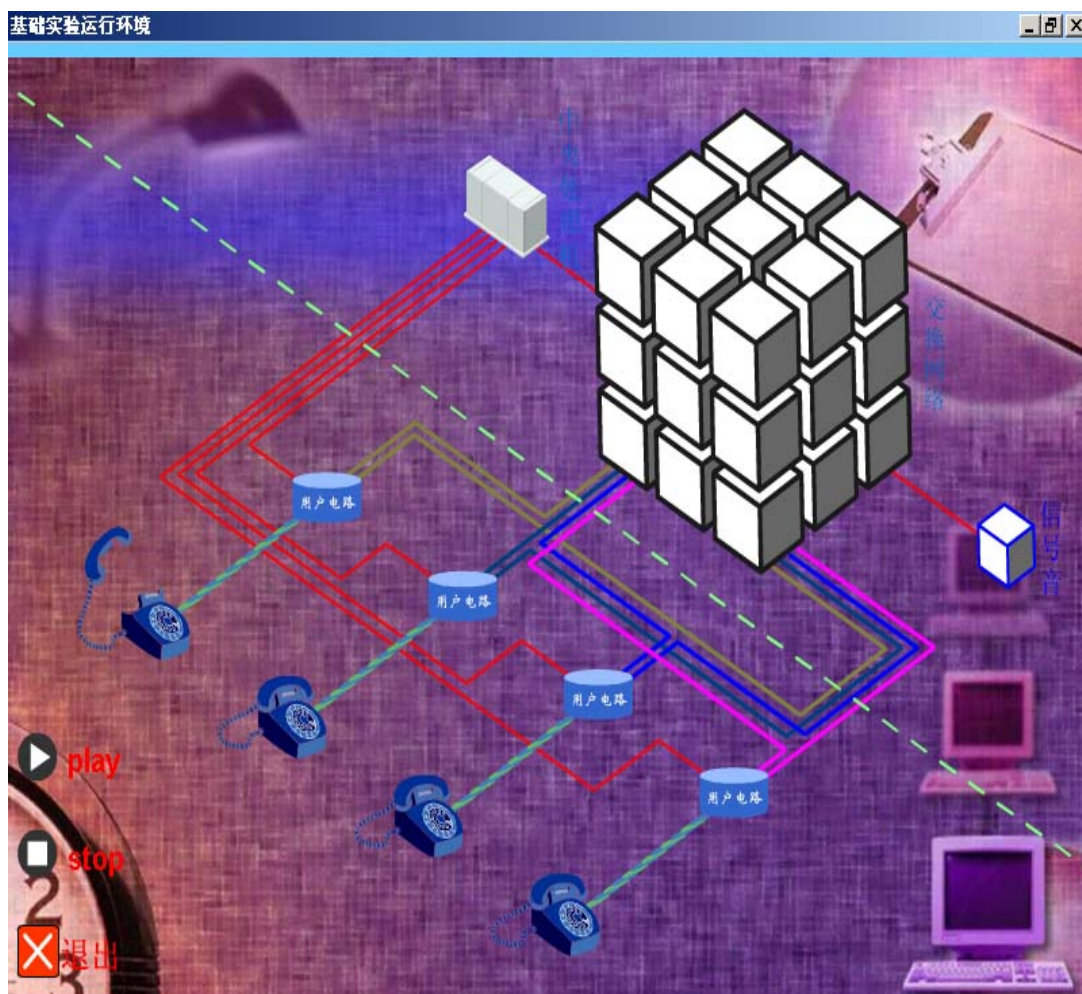
[上一页](#)

[下一页](#)

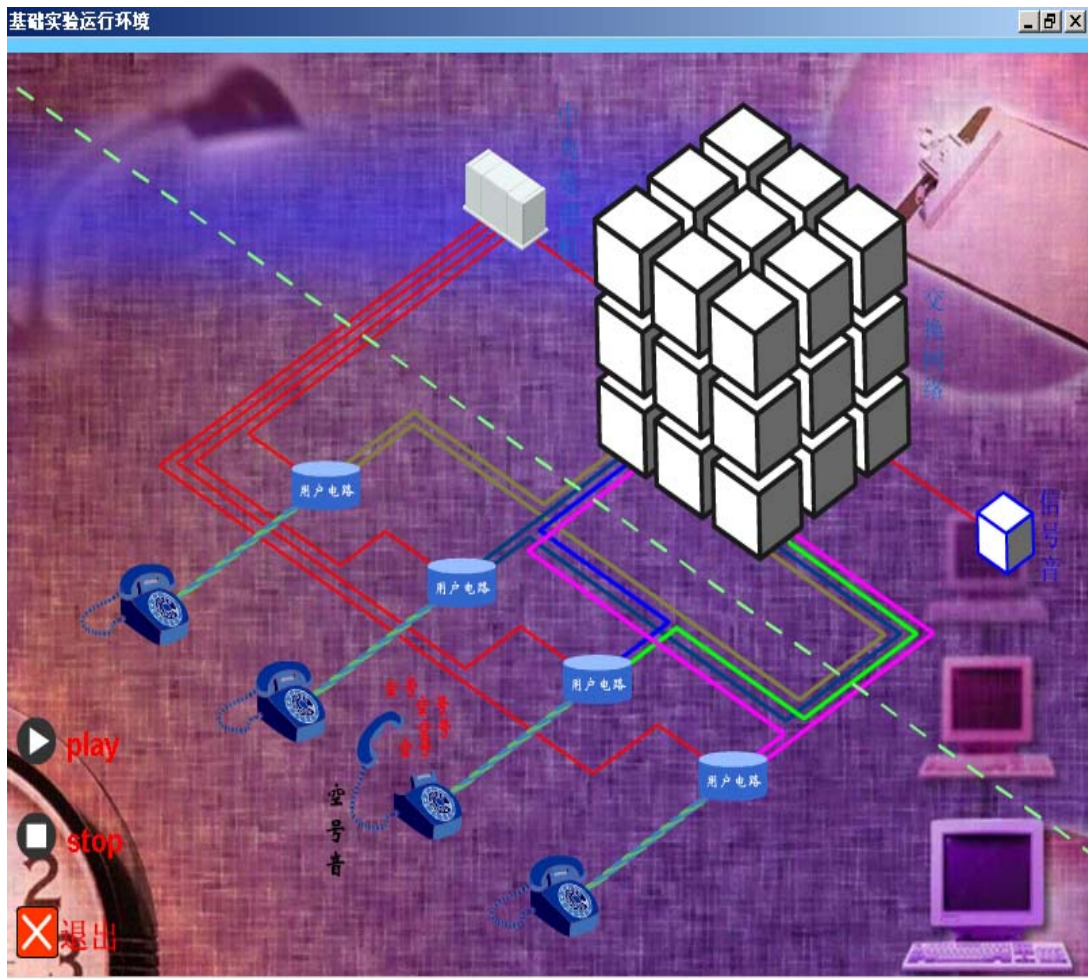
7.如果话机62280521在忙音超时之前挂机（再次点击该话机），忙音显示将消失；我们假设它没有挂机，忙音超时之后，将进入嚎鸣音状态：



8.如果仍不挂机，将进入嚎鸣音超时状态，这时嚎鸣音显示消失,这时只有用户挂机话机才能复原，否则该话机将一直持续。



9.还有一个空号音状态，这是在被叫号码是一个不存在的时候出现的，界面将显示空号音，我们在最初时还会听到空号音，如下图所示：



[上一页](#)

[下一页](#)

三. 基础实验的错误分析：

对于基础实验部分，我们预设定了五个需编程实验，让学生完成各自指定的逻辑功能。在某一个实验运行的时候，其它的模块我们使用的都是预定义好的模块，这样就能检测这个实验模块代码的正确性。在演示时，我们加载的都是预定义的模块，而不是加载任何一个学生编写的模块。

基础实验一：

时间表调度实验实现的是调度表的初始化，如果初始化不成功的话，可能会造成周期性调度程序的延迟或者频繁调用，对于前者可能的结果是检测不要需要的事件（例如脉冲、位间隔），对于后者，频繁的调度可能造成系统的负担过大。

基础实验二：

摘挂机检测实验实现的是一个200ms摘挂机检测的周期级扫描，如果该函数的编写不正确，就不能把正确的摘挂机事件放入摘挂机事件队列中，这样就不可能生成正确的主、被叫进程，从而导致不可能产生音信号、设定超时定时器等。举一个例子，如果该函数的实现为空的话会导致主叫点击话机后，听不到拨号音的情况；

基础实验三：

脉冲计数实验实现的是一个8ms脉冲检测的周期级扫描，如果该模块的编写不正确，就不会对一位号码按键产生的脉冲正确地计数，例如把号码“5”产生的5个脉冲识别出4个或者3个的情况。如果该函数的实现为空的话，即使位间隔识别的函数编写正确，在拨号音的情况下点击号码产生的脉冲也不能识别，从而使主叫进程得不到一个“送”上来的号码而导致拨号音超时，导致听到忙音。

基础实验四：

位间隔识别实验实现的是一个100ms位间隔检测的周期级扫描。它和脉冲计数模块的关系是：脉冲计数模块实现的是识别出各个脉冲，将脉冲计数；而位间隔识别模块是识别出一位号码的间隔，从而将对应线路的脉冲计数值发送给主叫进程。两个模块“合作”完成了一位号码的采集工作。

如果该函数的编写不正确，那么即使脉冲计数的函数编写正确，主叫进程仍收不到正确的一位号码；如果该模块的函数编写为空的话，会导致主叫进程收不到号码，使拨号音定时器超时出现忙音。

基础实验五：

软件送音实验实现的是把输入一种状态，确定合适的信号音。如果该模块的编写不正确的话，在运行时界面显示的信号音和听到的信号音是不一致的。如果需要事先听一下信号音，您可在安装目录下的“utility\ringsound”中找到相应的音信号文件。如果该模块的函数编写为空的话，会导致只有界面信号音提示，听不到任何的信号音。

基础实验六：

这是五个实验中最简单的一个，实现的是通信双方时隙的确定，只要在实隙表中自己时隙对应的位置添上对方的时隙号就完成了建立连接的功能。如果该模块的函数编写不正确，在连接时，界面显示就不会看到通话双方的线路相连 或者错误的线路相连；在拆除时，界面的连接显示不会消失。

分组实验一 分组交换演示实验

链接：[实验目的](#)

链接：[实验原理以及实验设计](#)

链接：[实验演示界面](#)

一.实验目的：

该部分实验动态演示了分组传输过程，包括X.25虚链路的建立、分组传输和X.25虚链路的拆除；主要体现了分组传输中面向连接的工作方式。

二.实验原理以及实验设计：

X.25建议是国际标准化组织（ISO）和国际电报电话咨询委员会（CCITT）制定的关于数据终端设备（DTE）和数据电路终接设备（DCE）之间的接口规程。

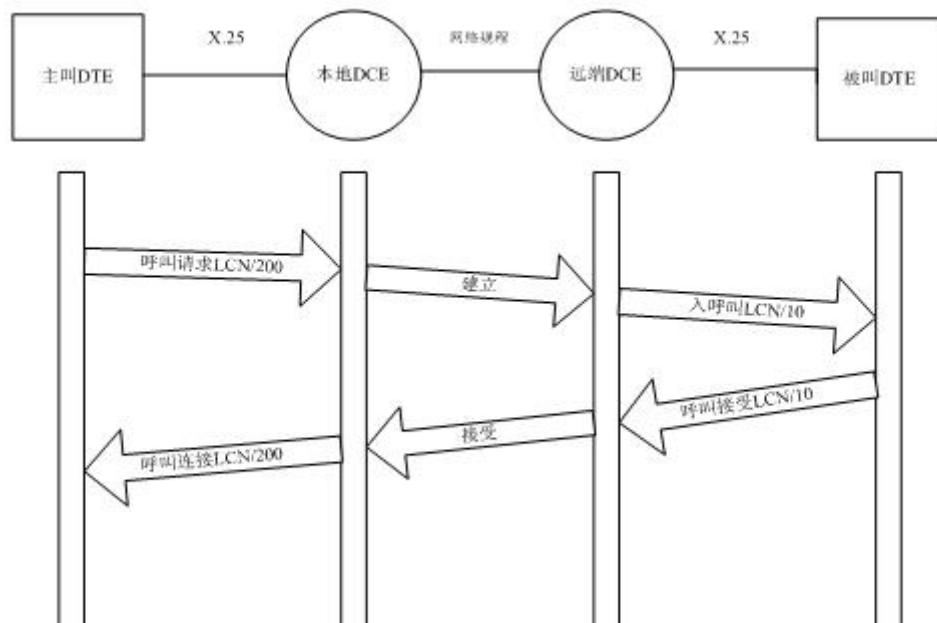
X.25标准的思路是为用户（DTE）和分组交换网络（DCE）之间建立对话和交换数据提供一些共同的规程，这些规程包括数据传输通路的建立、保持和释放，数据传输的差错控制和流量控制，防止网络发生阻塞，确保用户数据通过网络的安全，向用户提供尽可能多而且方便的服务。

X.25建议为分组交换网定义了开放系统互连（OSI）模型的下3层，这三层分别是：物理层，链路层，分组层。与OSI模型的下三层一一对应，只是OSI的网络层（第三层）改称分组层，其功能是一致的。X.25的三层协议为DTE——DTE之间的高层通信协议提供了可靠的基础。

X.25的分组层在X.25接口为每个用户呼叫提供一个逻辑信道（所谓的“呼叫”是指一次通信过程）。为每个用户的呼叫连接提供有效的分组传输，包括顺序编号，分组的确

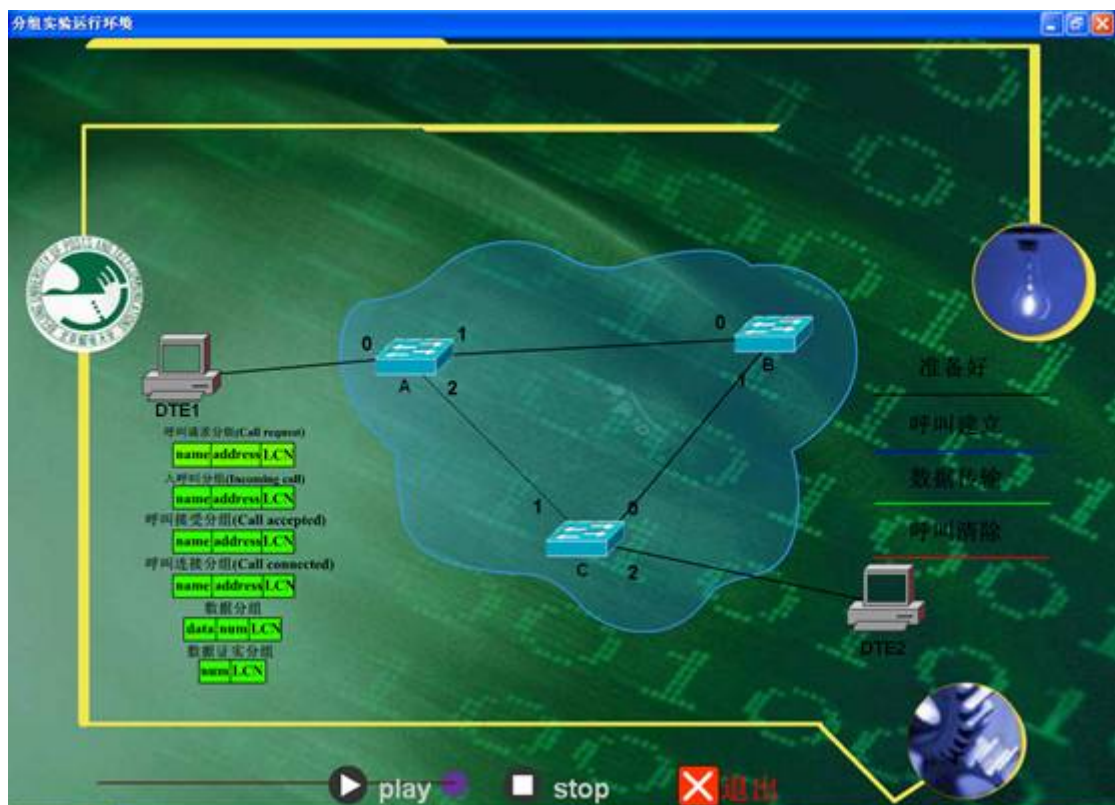
认和流量控制过程。提供交换虚电路（SVC）和永久虚电路（PVC）的连接。提供建立和清除交换虚电路的方法。

交换虚电路建立过程如下：



三.实验演示流程

实验界面如图所示：



DTE终端为DTE1～DTE2，分组交换机为Switcher_A～Switcher_C，网络拓扑如图所示。

实验演示分三个阶段虚电路连接建立阶段、数据传送阶段和连接拆除阶段。

1.连接建立阶段：

进入实验界面后，点击开始按钮，进入连接建立阶段。首先发起连接的终端发送“呼叫请求”（Call Request）分组，该“呼叫请求”分组包含可供分配的高端的LCN和被叫的DTE地址，启动一次呼叫。该分组发送到本地DCE，由DCE将该分组转换成网络规程格式，而且通过网络路由（交换）到远端DCE，由远端DCE将网络规程格式的呼叫请求分组转换为“入呼叫”（Incoming Call）分组，并发送给被叫的DTE，该分组包含了可供分配的低端的LCN。

被叫DTE通过发送“呼叫接受”(Call accepted)分组表示同意建立虚电路。远端DCE接收到“呼叫接受”分组之后，通

过网络规程传送到本地DCE，本地DCE发送“呼叫连接”(Call connected)分组到主叫DTE。主叫DTE接收到“呼叫连接”分组之后，表示主叫DTE和被叫DTE之间的虚呼叫已建立，可以进入数据传输阶段。DTE和DCE对应的逻辑信道就进入数据传输状态。

2.数据传输阶段：

链路连接建立完成以后，主叫数据的终端向被叫终端发送数据。分组交换机根据数据分组的LCN值，按照路由表转发分组。被叫终端接到数据后发送数据证实分组。

3.连接拆除阶段：

通信完毕后，在虚呼叫任何一端的DTE都能够清除呼叫，而且呼叫也可以由网络清除，我们这里以主叫方发送释放请求说明链路的拆除过程。呼叫清除的过程实验演示。主叫DTE发“清除请求”(Clear Request)分组，该分组通过网络到达远端DCE，远端DCE发“清除指示”(Clear Indication)分组到被叫DTE，被叫DTE用“清除证实”(Clear conform)分组予以响应。该“证实”传到本地DCE，本地DCE再发送“清除证实”到主叫DTE。完成清除规程之后，虚呼叫所占用的所有逻辑信道都成为“准备好”状态。

分组实验二 永久虚电路建立演示实验

链接：[实验目的](#)

链接：[实验过程](#)

一.实验目的：

本部分实验将模拟分组交换网中永久虚电路的实现。它根据X.25建议的分组层提供永久虚电路连接的原理，让学生动手在分组交换网中建立一条永久的虚连接。通过这个实验，学生可以进一步了解分组交换网所提供的面向连接的服务，以及分组交换网中的分组交换节点对网中传输的分组按路由表转发的原理。

二.实验过程：

实验界面如图所示：



进入实验时，分组交换机 A、B、C 附近对应可以填充的空路由表，DTE1 终端附近可以填充连接的目的地址和 DTE1 到分组交换节点 A 端的逻辑信道号。实验要求建立 DTE1 到 DTE2

的交换虚电路连接，学生可选择DTE1——A——B——C——DTE2或者DTE1——A——C——DTE2两条路由，并根据所选择路由填充路由表。路由表的填充尾端到端的一次填充。每个节点对路由表的范围都有限制，所填的路由值超出限制范围时，报错，并允许重新填写。例如，所有链路的逻辑信道号的范围是1——7，超出此范围应报错；DTE2的地址为“DTE2”，分组交换节点A、B和C所连接的端口号如图所示。填入的路由表正确时，进入数据传送阶段的演示过程，否则报错，学生重新填写。

分组实验三 分组交换编程实验

链接：[实验目的](#)

链接：[实验设计](#)

链接：[实验主要数据结构](#)

一.实验目的：

该部分实验是让学生进一步理解虚电路的建立、拆除以及路由表查询的过程。

二.实验设计：

该部分实验分三个实现函数，分别是虚电路的建立函数，虚电路的拆除，以及分组转发函数。

三.实验主要数据结构：

1．虚电路建立函数

函数功能：DTE终端线程调用虚电路建立函数，根据连接的目的地址，查询路由表，为本次连接分配端口号以及逻辑信道号，初始话虚电路的建立。

函数原型：

```
#include "pconstant.h"
#include "string.h"

extern "C" _declspec(dllexport) bool setup(char * address,
int port_lcn[2],DTERoutTbaleType DTERoutTbale[])
{
    return true;
}
```

主要数据结构说明：

函数参数说明

in (输入参数) char * address : 目的地址

out (输出参数) int
port_lcn[2] : 输出分配的
端口号，以及逻辑信道号
port_lcn[0]为端口号，
port_lcn[1]为逻辑信道号

in/out (输入/输出参数) DTERoutTbaleType
DTERoutTbale[] : 路由表

函数要求，根据目的地址，在给定的路由表中查询该目的地址对应的端口，然后在该端口上查找一个空闲的LCN分配给该连接，并将该LCN的状态置为忙。如果能够成功分配LCN返回true，否则返回false。

2. 虚电路拆除函数

函数功能说明：

DTE终端线程调用虚电路拆除函数来完成虚电路的拆除，

函数原型：

```
#include "pconstant.h"
```

```
#include "string.h"
```

```
extern "C" __declspec(dllexport) bool clean(char * address, int  
port_lcn[2], DTERoutTbaleType DTERoutTbale[])
```

```
{  
    return true;  
}
```

主要数据结构说明：

函数参数说明

in (输入参数) char * address : 目的地址

in (输出参数) int
port_lcn[2] : 输出分配的

端口号，以及逻辑信道号
port_lcn[0]为端口号，
port_lcn[1]为逻辑信道号

in/out (输入/输出参数) DTERoutTbaleType

DTERoutTbale[]：路由表

函数要求，根据目的地址，端口号和逻辑信道号，判断端口号是否在允许的范围内，然后检查对应的逻辑信道状态，如果为忙，将其置为闲，返回true，否则返回false

3．分组转发函数

函数功能说明:

DTE终端线程调用虚电路拆除函数来完成虚电路的拆除。

函数原型：

```
#include "pconstant.h"
```

```
extern "C" _declspec(dllexport) bool pack_switch ( int  
In_port_lcn[],int Out_port_lcn[], RoutTableType Table[])
```

```
{  
    return true;  
}
```

主要数据结构：

in (输入参数) int In_port_lcn[]：分组交换节点线程接到的分组入端的端口号和逻辑信道号，In_port_lcn[0]为端口号，In_port_lcn[1]为逻辑信道号。

out (输出参数) int Out_port_lcn[]：分组出端的端口号和逻辑信道号，Out_port_lcn[0]为端口号，Out_port_lcn[1]为逻辑信道号。

in (输入参数) RoutTableType Table[]：分组交换节点线程的路由表信息。

函数要求：分组转发函数根据分组入端的端口号和逻辑信道号，查找本分组交换节点线程的路由表，若查找到所需的路由信息（路由表数组总共有三项），将出端的端口号和逻辑信道号值赋给Out_port_lcn[]数组，函数返回true，否则函数返回值为false。

所需的头文件信息：

```
#define MaxPortNum    //DTE上的最多端口

#define MaxAddressNum //每个端口上对应的最大的目的地址

#define MaxLcnNum     //每个端口上最大的逻辑信道数

#define MaxRouterNum  //分组交换节点线程的路由表的行数

struct RoutTableType
{
    int  in_port;  入端口号
    int  in_lcn;   入LCN
    int  out_port; 出端口号
    int  out_lcn;  出LCN
};

struct DTERoutTbaleType
{
    char* Address[MaxAddressNum]; // 目的地址
    bool lcn_state[MaxLcnNum]; // LCN的状态，true 为忙，false为闲
};

DTERoutTbaleType DTERoutTbale[MaxPortNum]; // DTE的路由表
```

ATM实验一 ATM交换演示实验

链接：[实验目的](#)

链接：[实验原理及设计](#)

链接：[实验演示流程](#)

一.实验目的：

该部分实验动态演示了ATM的传输过程，包括ATM链路的建立、ATM信元传输和ATM链路的拆除；主要体现了ATM采用面向连接的工作方以及采用和固定长度信元的特点。

二.实验原理及设计：

ATM是面向连接的网络，在端到端的通信前必须建立连接。ATM网络通常有永久虚电路（PVC）和交换虚电路（SVC）两种方式。永久虚连接（PVC）是通过预定或预分配的方法建立的连接。交换虚连接是当用户需要使用网络资源时，才由网络动态分配，当呼叫结束时，SVC会被拆除，并可以分配给另一个用户。ATM网络的优点是能够根据需要动态建立与释放连接，用户可以根据不同应用的需要，在同一时间内建立多条SVC，支持多种服务，并允许较多的用户有效的使用网络资源。这里我们主要演示ATM的SVC建立、传输和释放过程。

SVC的建立、管理与释放过程就是ATM信令的交互过程。ATM网络有用户终端、专用ATM交换机和公用ATM交换机构成，SVC的实现涉及终端与交换机、交换机与交换机之间的信令。ATM信令主要分为UNI(用户－网络接口)信令和NNI（网络－网络接口）信令。

1.UNI信令

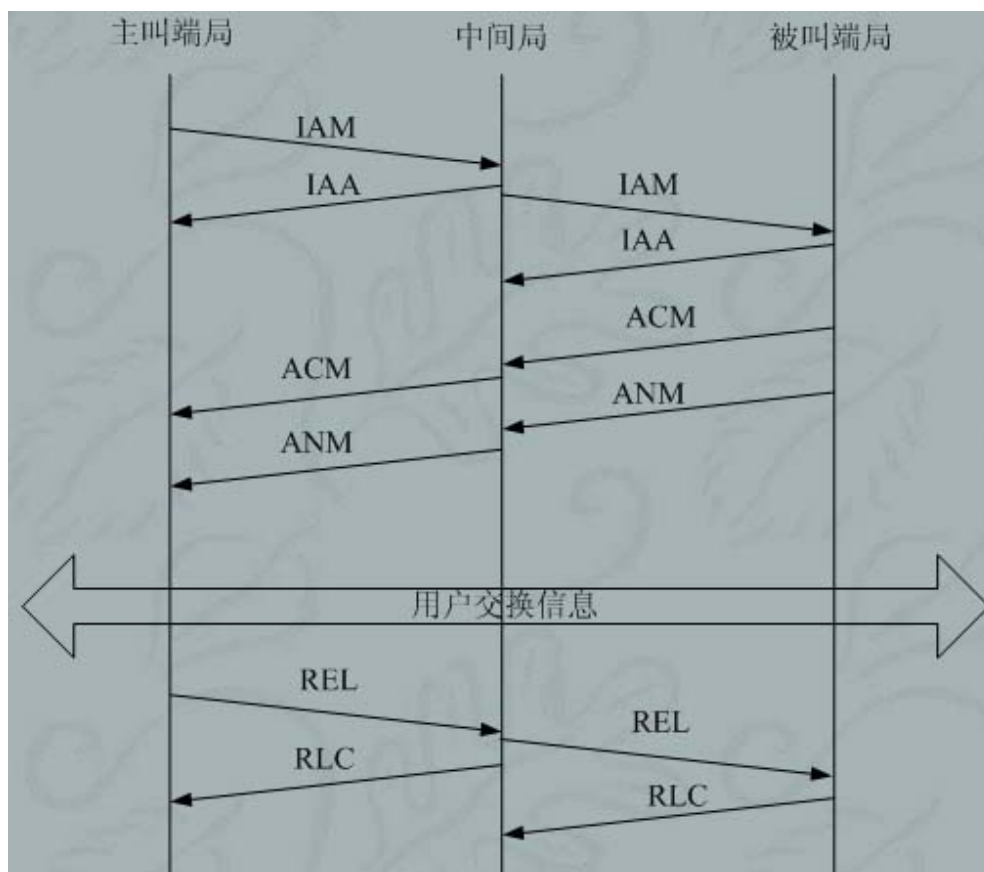
ATM UNI信令是在ATM网络中用户终端和网络之间进行连接的建立、释放和维护的协议。ITU-T规定了两种UNI

呼叫连接控制：点到点的呼叫连接控制和点到多点的呼叫连接控制。点到点的呼叫连接控制涉及到3个实体：主机方（用户终端）、网络侧被叫方（用户终端）。其一个点到点的呼叫连接的建立和释放过程如下图所示：



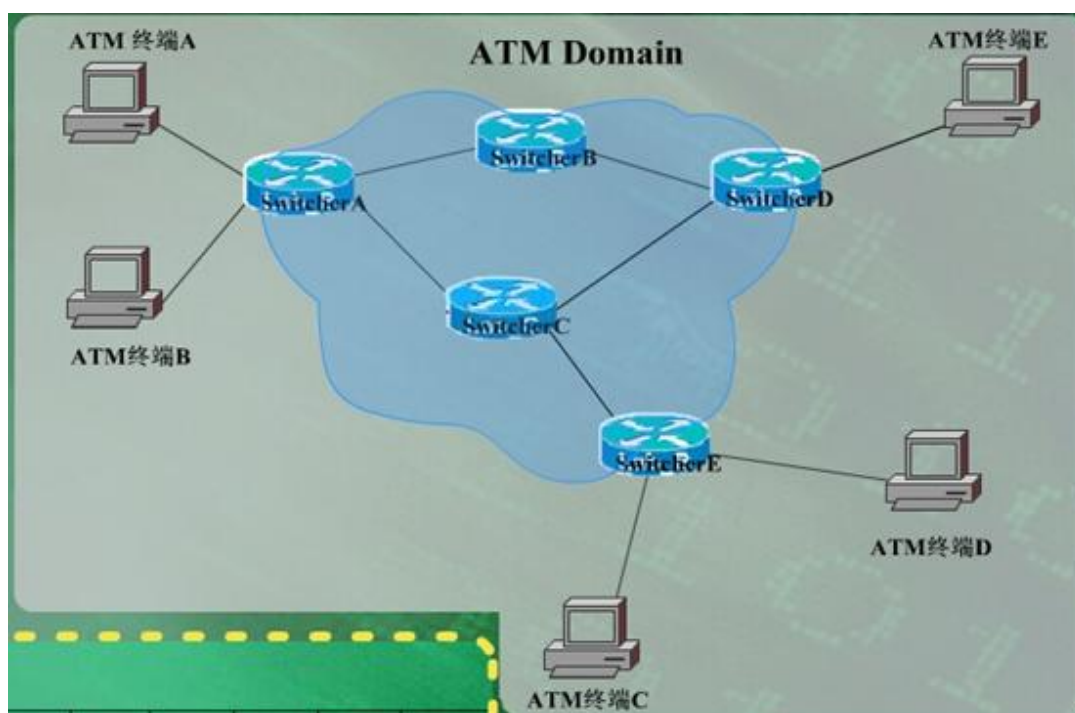
2. NNI信令

ATM网络局间信令采用网络节点接口（NNI）信令，NNI信令是基于现有的No.7信令的ISDN用户部分（ISUP描述和定义的，是ISDN NNI信令ISUP的扩充与增强。将ATM的NNI信令称为宽带综合业务数字网用户部分（B-ISUP）。ITU-T建议 Q.2761~Q.2764描述了NNI信令。下图所示为一个点到点的呼叫建立和拆除过程中的NNI信令操作。



三.实验演示流程

实验界面如图所示：



ATM终端为ATM_A ~ ATM_E，ATM交换机为Switcher_A ~ Switcher_E，网络拓扑如图所示。

该实验提供三种连接方式，分别是ATM_A——Switcher_A——Switcher_C——Switcher_E——ATM_C、ATM_A——Switcher_A——Switcher_B——Switcher_D——Switcher_C——Switcher_E——ATM_D和ATM_A——Switcher_A——Switcher_B——Switcher_D——ATM_E。ATM交换机端口标号在被鼠标选中时自动弹出。

实验演示分三个阶段ATM连接建立阶段、数据传送阶段和连接拆除阶段。

1.连接建立阶段：

进入实验界面后，点击开始按钮，然后按照提示，选择你想要建立连接的终端，进入连接建立阶段。首先发起连接的终端发送连接建立（SETUP）消息，启动一次呼叫。主叫端局接到SETUP消息后，向主叫方发送呼叫处理（CALL PROCEEDING）消息作为对SETUP消息的证实，该消息中包含网络侧分配给主叫的VPI、VCI值；向网络侧发送IAM消息。中间交换节点接收到IAM消息后，返回IAA作为确认，同时向被叫端局发送IAM消息；被叫端局接收到IAM消息后，同样返回IAA消息作为确认，同时向被叫终端发送SETUP消息。被叫用户使用呼叫处理（CALL PROCEEDING）表示被叫端正在处理来话呼叫，接着传送提醒（Alerting）消息，表示被叫终端正在提醒被叫用户有呼入。被叫端局收到被叫用户的通知消息后，向中间局发送地址收全消息ACM，中间端局将该消息转发给主叫端局。被叫用户最后发送接受呼叫（Connect）消息，被叫端局发送ANM消息，将呼叫建立消息传送给相应的节点。主叫端局接到ACM消息后向主叫用户回送提醒（Alerting）消息以通知主叫用户；接到ANM消息后通过连接消息（Connect）通知主叫用户进入连接阶段，主叫用户返回连接证实（CONNECT ACK）消息，进入信息传送阶段。

2.数据传送阶段：

链路连接建立完成以后，系统向用户提示连接建立完成，用户点击已经建立连接的终端，进入数据传输阶段。首先在发送数据的终端演示ATM数据从用户层到ATM适配层再到ATM层，最后封装成53个比特的ATM信元的过程。ATM交换机根据ATM信元的信头所带的VPI和VCI值，按照路由表转发信元。这里动态演示了ATM信元信头的替换过程。在数据接收终端，这里演示了ATM信元经过由底层向高层的拆分合并，最后得到用户数据的过程。

3.连接拆除阶段：

通信完毕后，主被叫用户都可发送连接拆除请求，我们这里以主叫方发送释放请求说明链路的拆除过程。首先ATM主叫方发送释放消息（RELEASE）消息。主叫端局回送释放结束（RELEASE COMPLETE）消息确认网络侧完成释放操作；然后向中间端局发送释放消息（REL），请求释放连接，中间端局接到返回释放证实（RLC）作为响应并将此消息往下传送，同时拆除请求释放连接的链路。ATM网络的拆线过程是在传送REL消息的同时逐段释放链路。被叫端局接到释放请求消息（REL）后，向被叫用户发送释放消息（RELEASE），被叫用户回送释放完毕消息（RELEASE COMPLETE）确认释放消息（RELEASE）。释放网络到被叫端局的连接。整个通信过程结束。

ATM实验二 ATM路由表查询实验

链接：[实验目的](#)

链接：[实验原理以及实验设计](#)

链接：[实验主要数据结构](#)

一.实验目的：

该部分实验是让学生理解ATM交换机根据路由表转发ATM信元的过程。

二.实验原理以及实验设计：

ATM是面向连接的网络，在端到端的通信前必须建立连接。ATM连接建立以后，在ATM交换节点保存着ATM连接信息的路由表。ATM交换实际上完成ATM信元的选路、信头翻译与排队的3个基本功能。选路就是选择物理端口的过程，即从某个入线端口交换到某个出线端口的过程。信头翻译是指将信元的信头值（入VPI/VCI）变换为输出信头值（出VPI/VCI）的过程。信头翻译与选路功能合作共同完成ATM交换。信头翻译和选路功能的实现是根据翻译表进行的，这里翻译表就是ATM连接建立后保存在ATM交换节点的路由表。这里我们设计了一个查询路由表的实验。

三.实验主要数据结构：

函数功能：在一个给定的路由表中，设计一个根据入端口、入VPI和入VCI查询出端口、出VPI和出VCI的函数

函数原型：

```
#include "aconstant.h"
```

```
extern "C" _declspec(dllexport) int Atm_switch ( int  
in_port,int in_vpi, int in_vci ,RoutTableTypeItem  
RoutTable[])
```

```
{  
}
```

主要数据结构说明：

头文件“aconstant.h”信息

```
#define Max_Rout_Line 5
```

//每个路由表最大的表项数。

```
struct RoutTableTypeItem
```

```
{
```

```
    int  in_port;
```

```
    int  in_vpi;
```

```
    int  in_vci;
```

```
    int  out_port;
```

```
    int  out_vpi;
```

```
    int  out_vci;
```

```
};
```

//每一个表项记录入出路由信息

函数参数说明int in_port,入端口

int in_vpi,入VPI

int in_vci ,入VCI

RoutTableTypeItem RoutTable[]路由表

函数要求，根据参数提供的信息，在给定的路由表中查询出端口和出VPI/VCI，返回该项信息所在路由表数组的下标，若找不到则返回-1。

MPLS实验一 MPLS演示实验

链接：[实验目的](#)

链接：[演示过程](#)

一.实验目的：

在这一实验中，主要通过不同角度的演示，让学生熟悉和了解MPLS交换技术的基本原理。

二.演示过程：

实验界面如图所示：



终端主机为Local_hostA1和Remote Host_B1。路由器为：入口LSR_A，核心LSR_B、LSR_C、LSR_D，以及出口LSR_E。网络拓扑结构如图所示。右边是路由表，左下方是标记转发表。鼠标放在各路由器上时就会显示各端口号。

实验演示分为三个阶段：建立连接阶段、数据传输阶段和拆除连接阶段。

1. 建立连接阶段：

进入实验界面后，点击开始按钮，进入建立连接阶段。在LDP协议控制下，进行MPLS节点间标记分配与分发，从而建立LSP路径（采用下游按需标记分发和有序的标记分发控制模式）。

界面如图所示：



2. 数据传输阶段：

在LSP路径建立完成后，点击继续按钮，进入数据传输阶段。此时演示分组在MPLS域中基于标记进行转发的过程。

界面如图所示：



3. 拆除连接阶段：

演示拆除连接也就是标记取消的过程。此阶段主要是通过文字方式解释连接拆除的过程。点击finish按钮即完成此阶段的演示过程。

界面如图所示：

拆除连接

因为MPLS网络中的虚连接也就是LSP路径是由标记拼接形成的，所以连接的拆除也就是标记的取消。广泛使用的标记取消的方式有两种。

其中一种是采用计时器的方式，即分配标记的时候为标记确定一个生存时间，并将生存时间与标记一同分发给相邻的LSR，相邻的LSR设定定时器对标记计时。如果在生存时间内收到此标记的更新信息，则标记依然有效并更新定时器；否则，标记将被取消。

另一种就是不设置定时器，这种方式下LSP要被明确地拆除，网络中拓扑结构发生变化（例如某目的地址不存在或者某LSR的下一跳发生变化等等）或者网络某些链路出现故障等原因，可能促使LSR通过LDP消息取消标记，拆除LSP。

finish

MPLS实验二 标记转发表实验

链接：[实验目的](#)

链接：[演示过程](#)

一.实验目的：

在这一实验中，主要让学生通过填表练习来独立控制为数据流分配，分发标记，建立LSP，并控制分组进行标记转发，从而加深对MPLS技术的理解。

二.演示过程：

实验界面如图所示：



点击“开始”按钮，进入此实验。会看到与“实验一”一样的请求分组传送过程。当请求分组到达出口LSR时，会要求同学填标记

转发表的各表项（目的地址前缀、入端口、出端口、入标记和出标记）。其中标记表项对输入有一定限制，只允许分配1~9标记值，填写超出范围时，会报错。填写时如果表项中是“-”，则此项不必填写。当同学完成标记转发表填写时，点击确定按钮提交所填表项。如果所填表项正确，则会按同学所分配的标记填写路由器中的标记表，然后会要求同学填写下一个路由表的标记转发表。如果所填表项不正确，则会报错，同学重新填写。

当同学正确填写完标记转发表后，点击“继续”按钮，进入数据传输阶段，这时会要求同学按照标记转发表填写标记分组中的标记号。如下图所示：



MPLS实验三 MPLS编程实验

链接：[实验目的](#)

链接：[实验设计](#)

链接：[实验主要数据结构](#)

一.实验目的：

安排的三个编程实验主要用于加强学生对MPLS交换中标记请求、标记分配与分发、标记分组转发的理解。

二.实验设计：

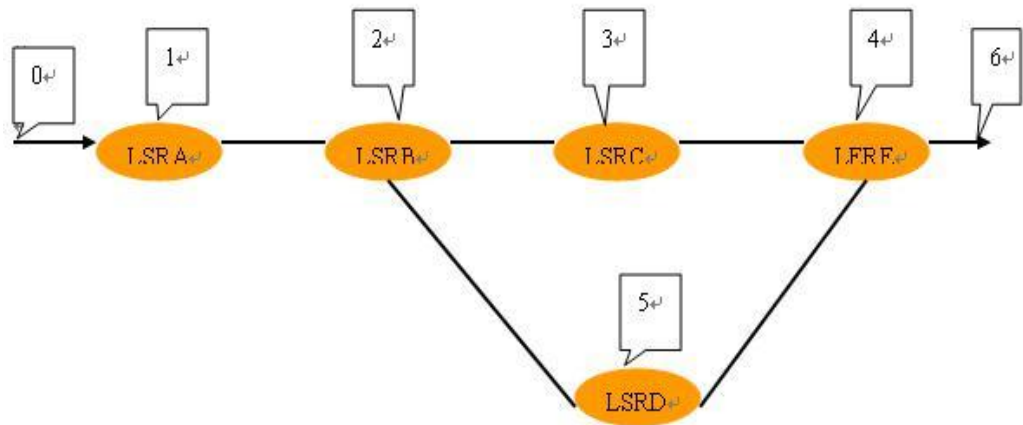
多协议标记交换MPLS（Multiple Protocol Labeled Switching）技术是将第二层交换和第三层路由结合起来的一种L2/L3集成数据传输技术。MPLS是一项面向连接的交换技术，因此有建立连接的过程。各个MPLS设备运行路由协议，在标记分发协议LDP的控制下根据计算得到的路由在相邻的路由器进行标记分配和分发，从而通过标记的拼接建立起从网络入口到出口的标记交换路径LSP。

在数据转发过程中，入口标记路由器LER根据数据流的属性比如网络层目的地址等将分组映射到某一转发等价类FEC，并为分组绑定标记。核心标记交换路由器LSR只需根据分组中所携带的标记进行转发即可。出口标记路由器LER弹出标记，根据分组的网络层目的地址将分组转发到下一跳。MPLS节点（MPLS标记交换路由器LSR或MPLS边缘路由器LER）均要创建和维护传统的路由表和标记信息库LIB。

路由表记录记录路由信息，用于转发网络层分组和标记分发从而建立标记交换路径。LIB记录了本地节点分配的标记与从邻接MPLS节点收到的标记之间的映射关系，用于标记分组的转发。

MPLS技术的核心实质在于：（1）网络中分组基于标记的转发（2）LDP协议控制下的进行标记分发从而建立标记交换路径LSP。

实验网络的拓扑结构（节点分布示意图）：



三.实验主要数据结构：

所需要的头文件：“mplsconstant.h”

其中的主要数据结构为：

//发送的请求信息包数据结构

```
struct ReqType
```

```
{
```

```
    int iFirstNode;    //请求信息包的源节点
```

```
    int iEndNode;      //请求信息包的目的节点
```

```
    double ipaddress;  //请求信息包包含的网络层目的IP地址前缀
                      // (例如197.42)
```

```
};
```

//路由表表项的数据结构

```
struct routertype
```

```
{
```

```
    double ipaddress; //网络层目的地址前缀
```

```
    int nexthop;      //下一跳节点
```

```
    int lasthop;      //上一跳节点
```

```
    int inpoint;      //入端口号
```

```
    int outpoint;     //出端口号
```

```
};
```

//标记信息表表项的数据结构

struct libtype

{

double ipaddress; //网络层目的地址前缀

int inpoint; //入端口号

int outpoint; //出端口号

int inlabel; //入标记值

int outlabel; //出标记值

};

//发送的标记信息包数据结构

struct LabelPack

{

int iFirstNode; //源节点号

int iEndNode; //目的节点号

int labelvalue; //标签值

};

struct funcusedtype

{

struct libtype libinfo; //包含的标记信息表项

struct LabelPack labelinfo; //包含的标记信息包数据结构

};

//发送的标记分组信息包类型

struct LabelledDataPack

{

int iFirstNode; //源节点号

int iEndNode; //目的节点号

struct MessageType DataInfo; //包含的标记分组类型信息

};

//标记分组类型

```

struct MessageType
{
    double ipaddress; //网络层目的地址前缀
    int labelvalue;   //输出标签值
};

```

1：标记请求实验要求函数：

```

extern "C" _declspec(dllexport) struct ReqType req_process(int
idnow,struct routertype routenow)
{
    struct ReqType reqtemp;
    return reqtemp;
}

```

参数意义：

int idnow：当前的节点号；

struct routertype routenow：当前所指的路由表的表项；

函数要求：根据提供的当前节点号和路由表表项值产生标记请求包；

过程描述：

标记请求包的源节点号由当前节点号提供，目的节点号和ip地址前缀由当前所指的路由表表项的下一跳节点和ip地址前缀提供；

2：标记分配与分发实验：

```

extern "C" _declspec(dllexport) struct funcusedtype
label_process(struct routertype routenow,int labelout,int idnow)
{
    struct funcusedtype tempstruct;
    return tempstruct;
}

```

参数意义：

struct routertype routenow：当前所指的路由表表项；

int labelout：分配的输出标签号；

int idnow：当前的节点号；

函数要求：

该函数要求根据提供的路由表当前表项、分配的输出标签号和当前节点号，构造一funcusedtype信息包。注：各节点的输入标签可以自由选定，但必须是1-9的整数；

过程描述：

该funcusedtype信息包的libinfo部分可由当前的路由表表项、当前分配的标签号的有关部分构成；labelinfo部分由当前节点号和当前的路由表表项的有关部分构成；

3．标记分组转发实验

```
extern "C" _declspec(dllexport) struct LabelledDataPack  
pack_process(struct routertype routenow,struct libtype libnow,int  
idnow)  
{  
    struct LabelledDataPack packtemp;  
    return packtemp;  
}
```

参数意义：

struct routertype routenow：当前所指的路由表表项；

struct libtype libnow：当前的标签信息表表项；

int idnow：当前的节点号；

函数要求:

该函数要求根据提供的路由表表项、标签信息表表项和当前节点号，构造出一个标签数据信息包。

过程描述：

该标签信息包的源节点、目的节点、IP地址前缀和标签值均可由当前节点号、路由表表项和标签信息表表项构成；